

Chapter 3 – Instruction-Level Parallelism and its Exploitation (Part 1)

ILP vs. Parallel Computers

Dynamic Scheduling (Section 3.4, 3.5, and Appendix C)

Dynamic Branch Prediction (Section 3.3, 3.9, and Appendix C)

Hardware Speculation and Precise Interrupts (Section 3.6)

Multiple Issue (Section 3.7)

Static Techniques (Section 3.2, Appendix H)

Limitations of ILP

Multithreading (Section 3.11)

Review

Purposes of MIPS General Registers (32-bit)

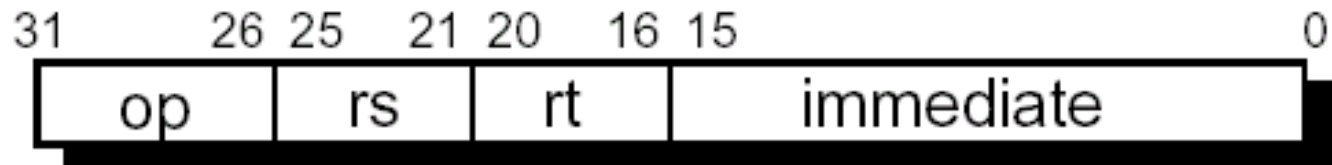
#	R Name	Use	#	R Name	Use
0	\$zero	Constant 0	24-25	\$t8-\$t9	temporaries
1	\$at	Assembler temporary	26-27	\$k0-\$k1	Reserved for OS kernel
2-3	\$v0-\$v1	Values for function returns and expression evaluation	28*	\$gp	Global pointer
4-7	\$a0-\$a3	Function arguments	29*	\$sp	Stack pointer
8-15	\$t0-\$t7	Temporaries	30*	\$fp	Frame pointer
16-23*	\$s0-\$s7	Saved temporaries	31*	\$ra	Return address

* Need to save when making procedure calls

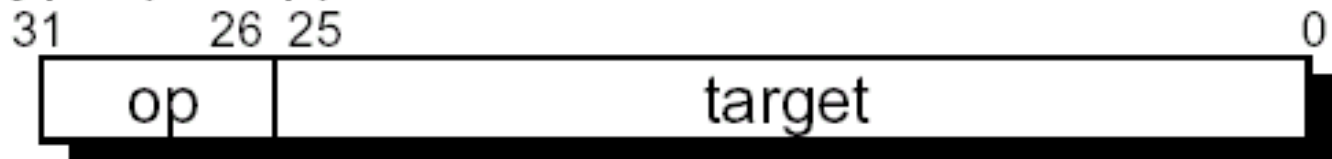
Review

Three Types of The MIPS Instruction

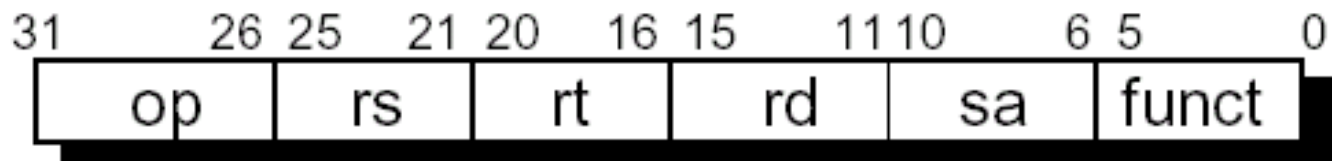
I-Type (Immediate)



J-Type (Jump)



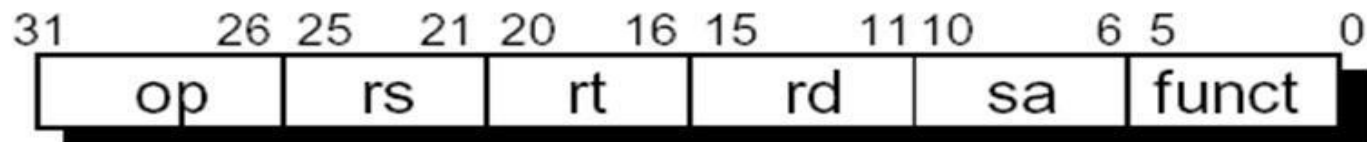
R-Type (Register)



Review

R-Type

R-Type (Register)



Example:

`add $t0, $t1, $t2` $\# \$t0 = \$t1 + \$t2$

`sub $t0, $t1, $t2` $\# \$t0 = \$t1 - \$t2$

`and $t0, $t1, $t2` $\# \$t0 = \$t1 \& \$t2$

0	9	10	8	0	0x24
---	---	----	---	---	------

Type R \$t1 \$t2 \$t0 and

Review

R-Type

Example:

`sll $t2, $s0, 8` `#$t2 = $s0 << 8 bits`

`srl $t2, $s0, 8` `#$t2 = $s0 >> 8 bits`

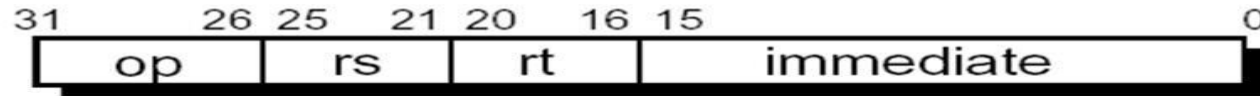
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
0		16	10	8	0x00

`shmat`

Review

I-Type: Load/Store

I-Type (Immediate)

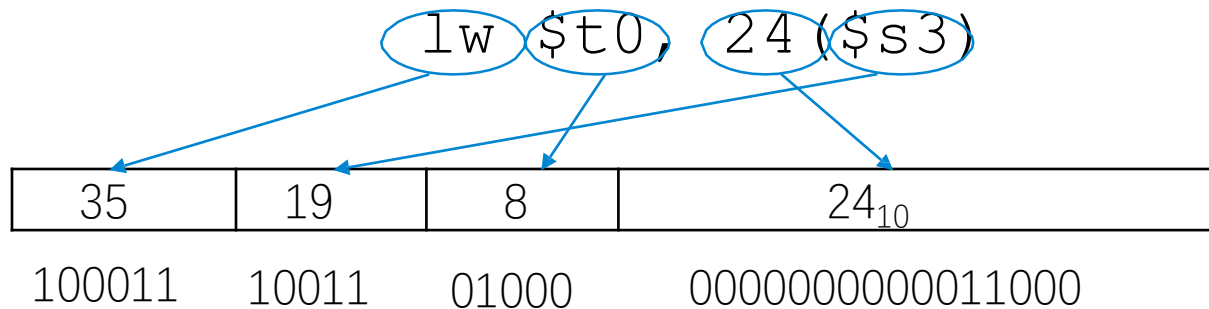


▪ **lw** **\$t0, 24 (\$s3)**

▪ **sw** **\$t0, 28 (\$s3)**

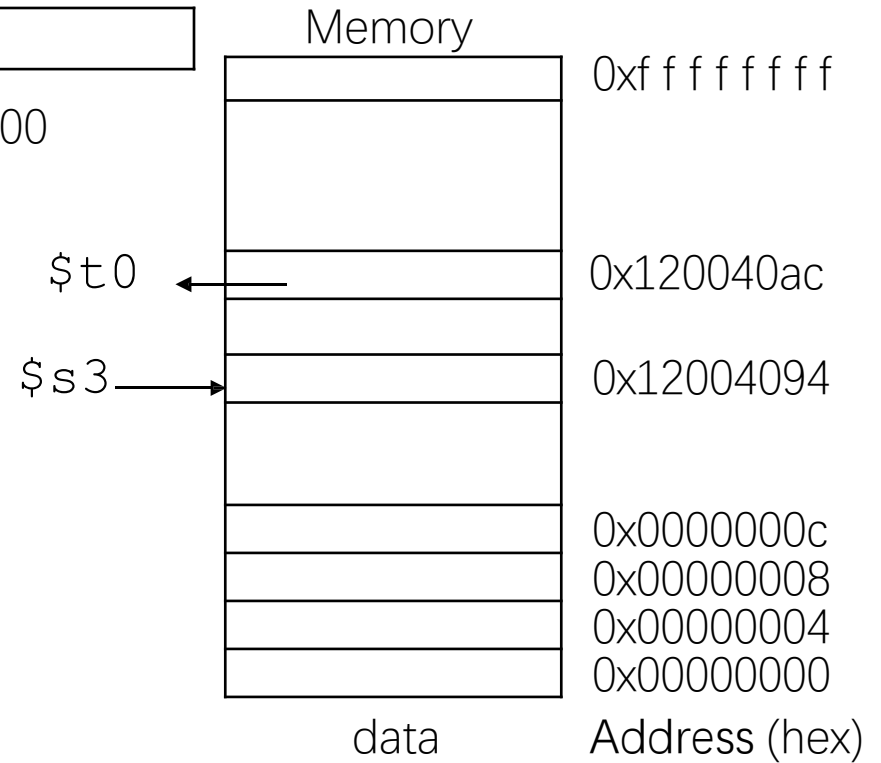
#load word from memory

#store word to memory



$$\begin{aligned} \$s3 + 24_{10} &= 0x12004094 + \mathbf{0x18} \\ &= \mathbf{0x120040ac} \end{aligned}$$

$$\begin{array}{r} \dots 0001\ 1000\ (\mathbf{0x18}) \\ + \dots 1001\ 0100\ (\mathbf{0x94}) \\ \hline \dots 1010\ 1100\ (\mathbf{0xac}) \end{array}$$



Review

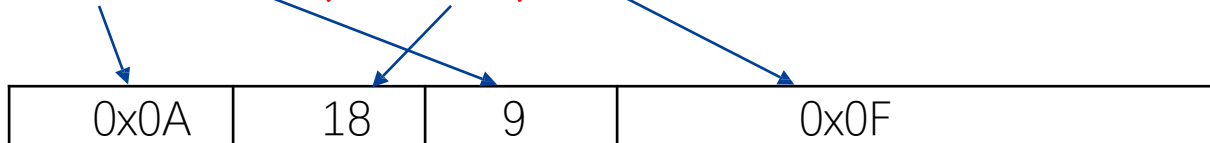
I-Type

Example:

`andi $t0, $t1, 0xFF00` `#$t0 = $t1 & ff00`

`addi $sp, $sp, 4` `#$sp = $sp + 4`

`slti $t1, $s2, 15` `#$t1 = 1 if $s2 < 15`



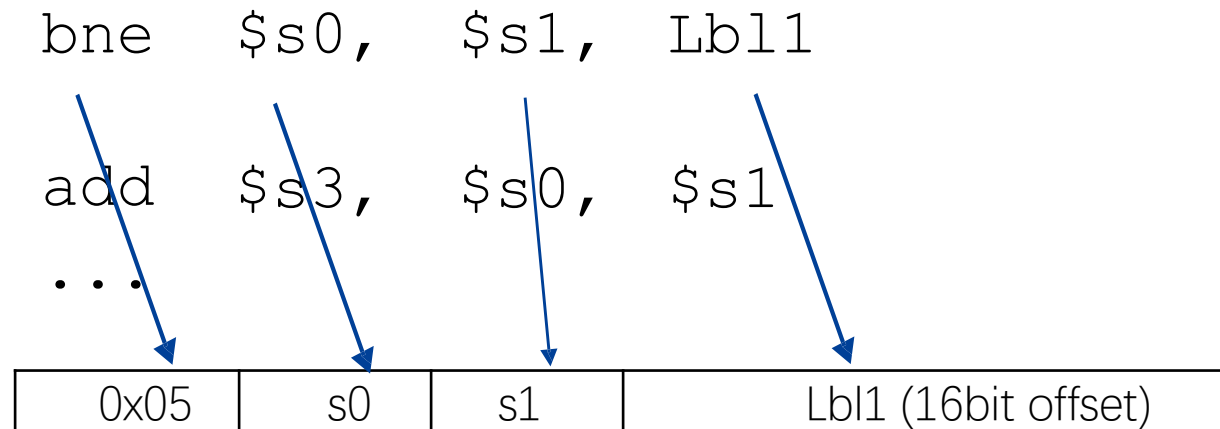
Immediate Op: $+2^{15}-1$ to -2^{15}

Review

I-Type

bne \$s0, \$s1, Lbl1 #go to Lbl1 if \$s0≠\$s1
beq \$s0, \$s1, Lbl1 #go to Lbl1 if \$s0=\$s1

Example: if (i==j) h = i + j;

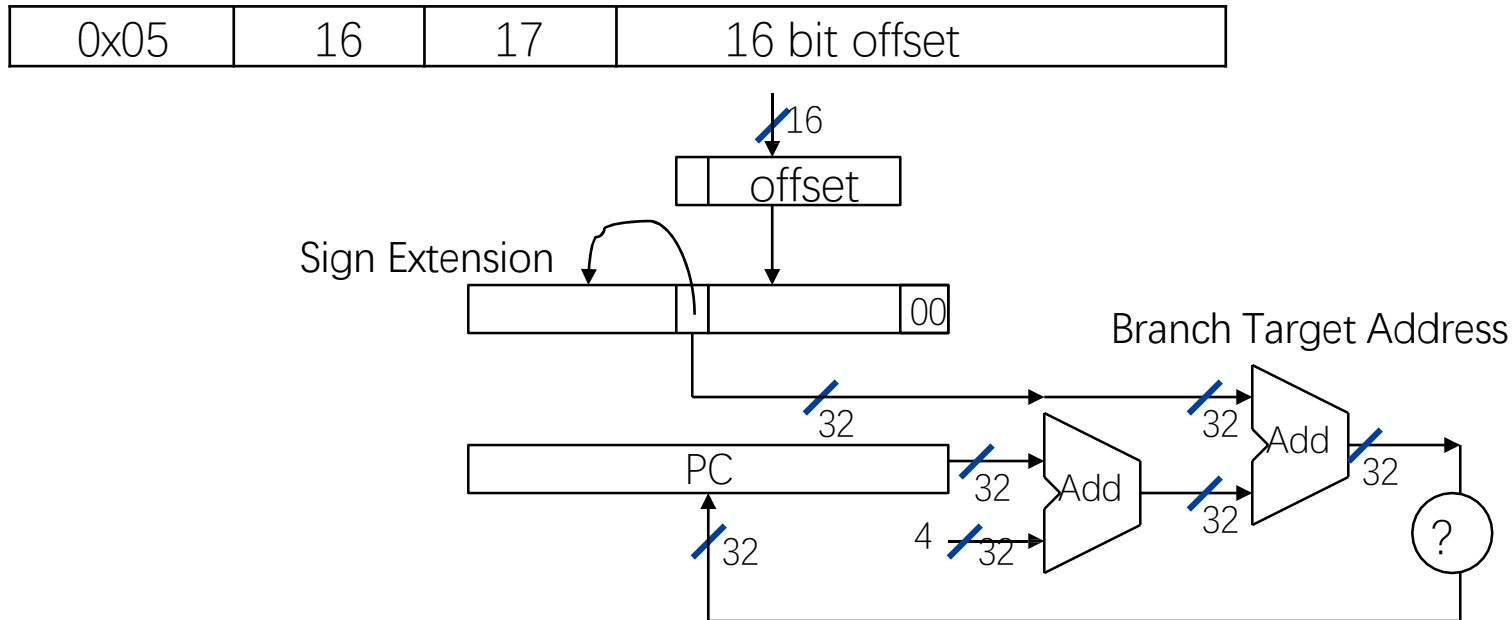


Immediate Op: $+2^{15}-1$ to -2^{15} Words = $+2^{17}-1$ to -2^{17} Bytes

Review

I-Type

bne \$s0, \$s1, Lb11 #go to Lb11 if \$s0≠\$s1



Immediate Op: $+2^{15}-1$ to -2^{15} Words = $+2^{17}-1$ to -2^{17} Bytes

Review

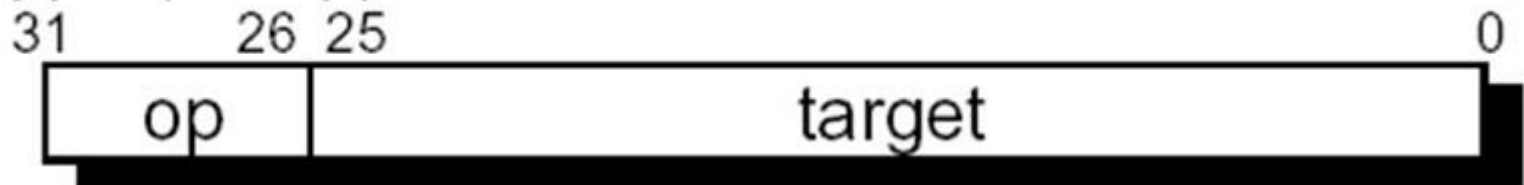
J-Type

Instruction Examples	Operations	Purposes
j 10000	go to 10000	Jump to 10000
jr \$31	go to \$31	Jump to \$31
jal 10000	\$31=PC+4; go to 10000	Jump&Link

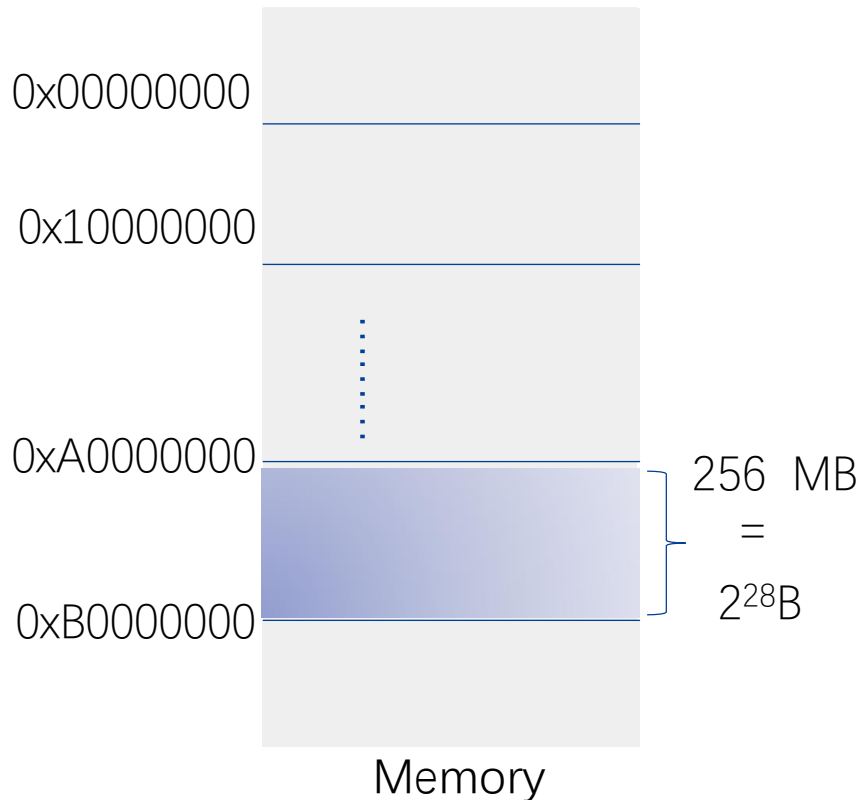
Review

J-Type

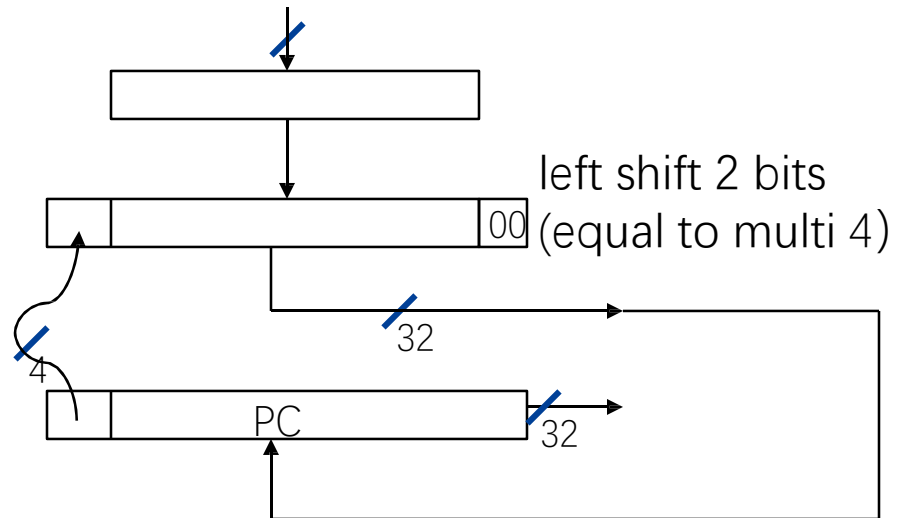
J-Type (Jump)



Address



26-bit address



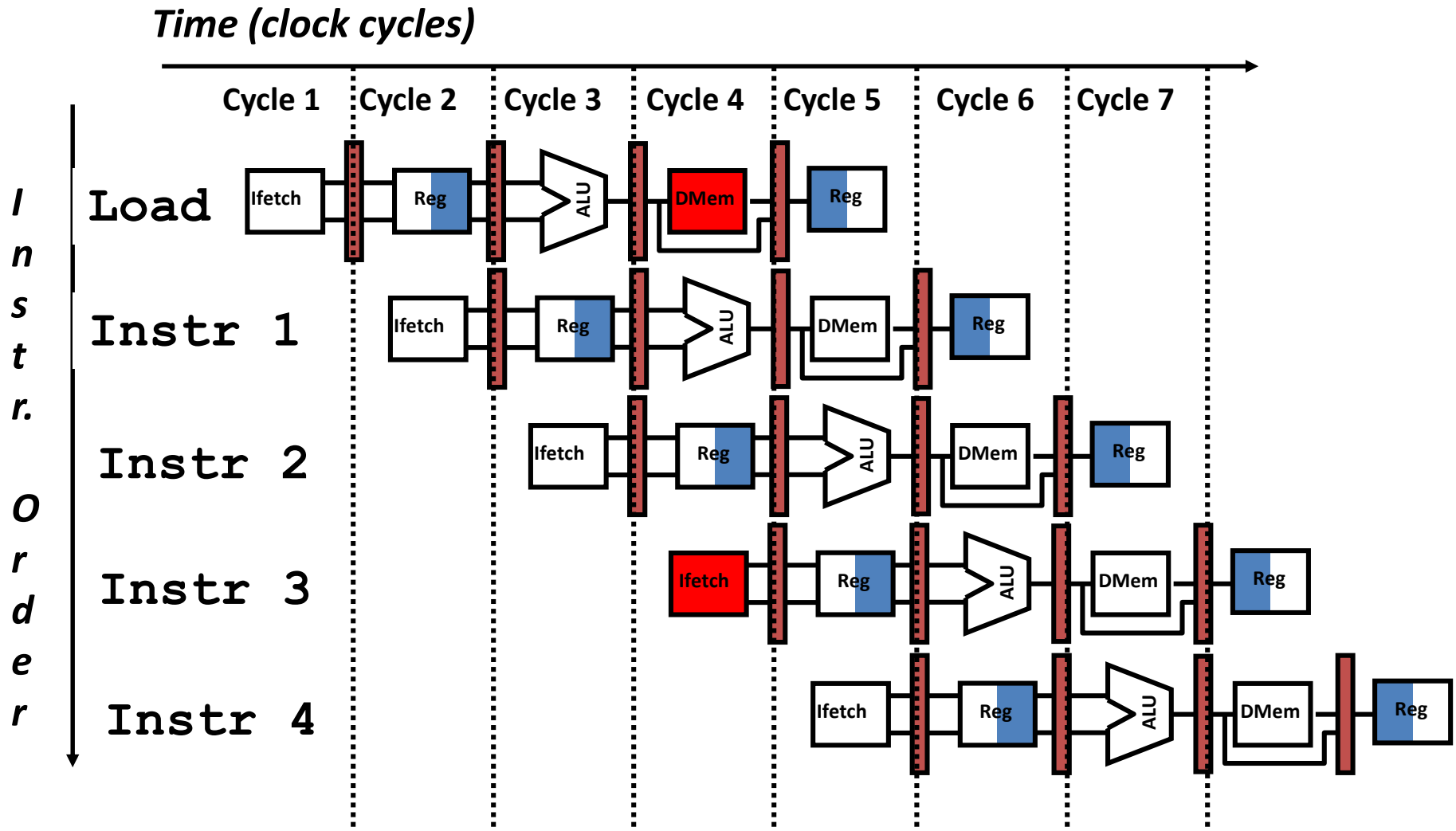
Review

Hazard: prevent next instruction from executing during its designated clock cycle

- **Structural hazards:** HW cannot support this combination of instructions (single person to fold and put clothes away)
- **Data hazards:** Instruction depends on result of prior instruction still in the pipeline (missing sock)
- **Control hazards:** Caused by delay between the fetching of instructions and decisions about changes in control flow (branches and jumps).

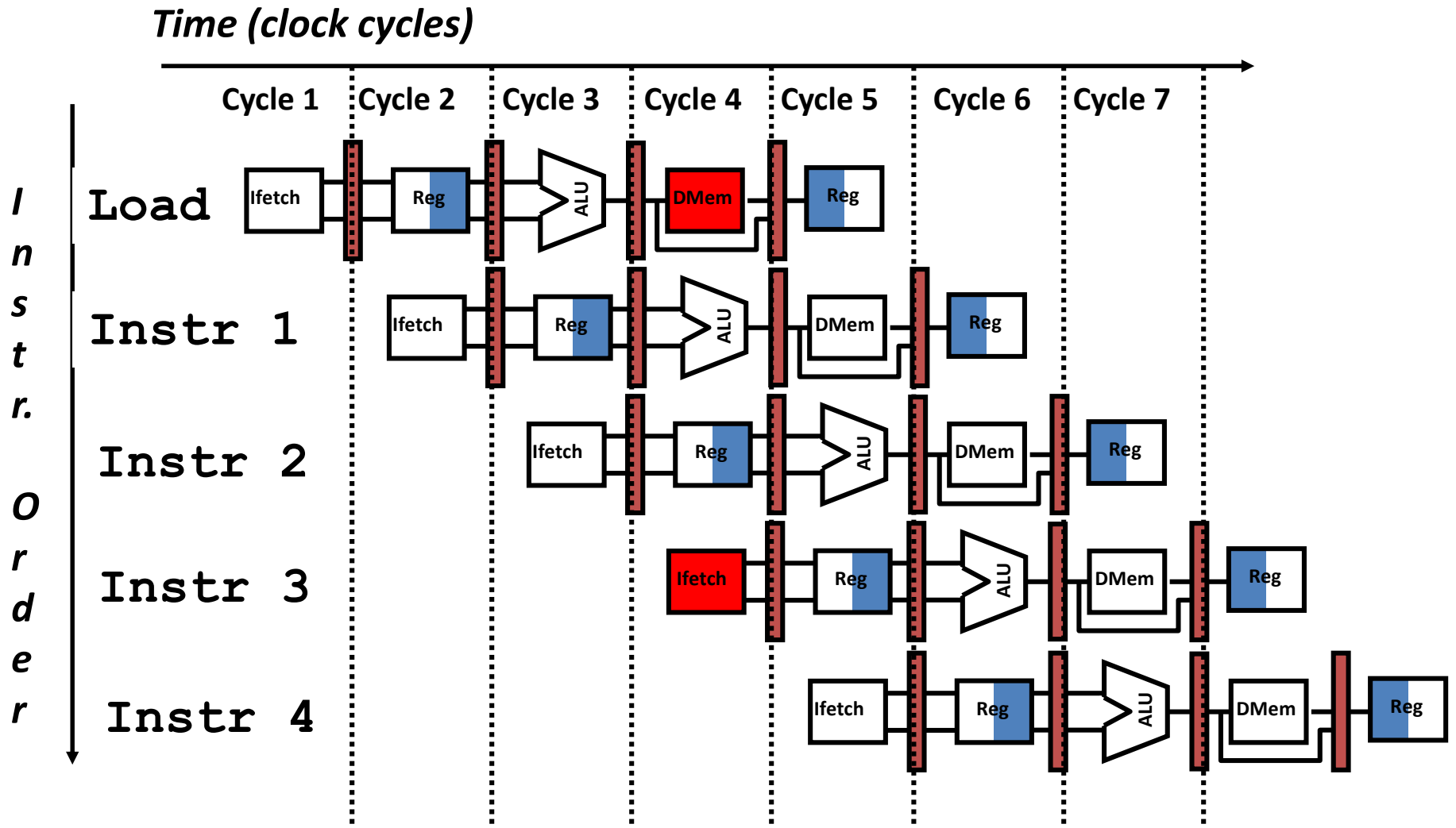
Review

- Structural hazards:** HW cannot support this combination of instructions (single person to fold and put clothes away)



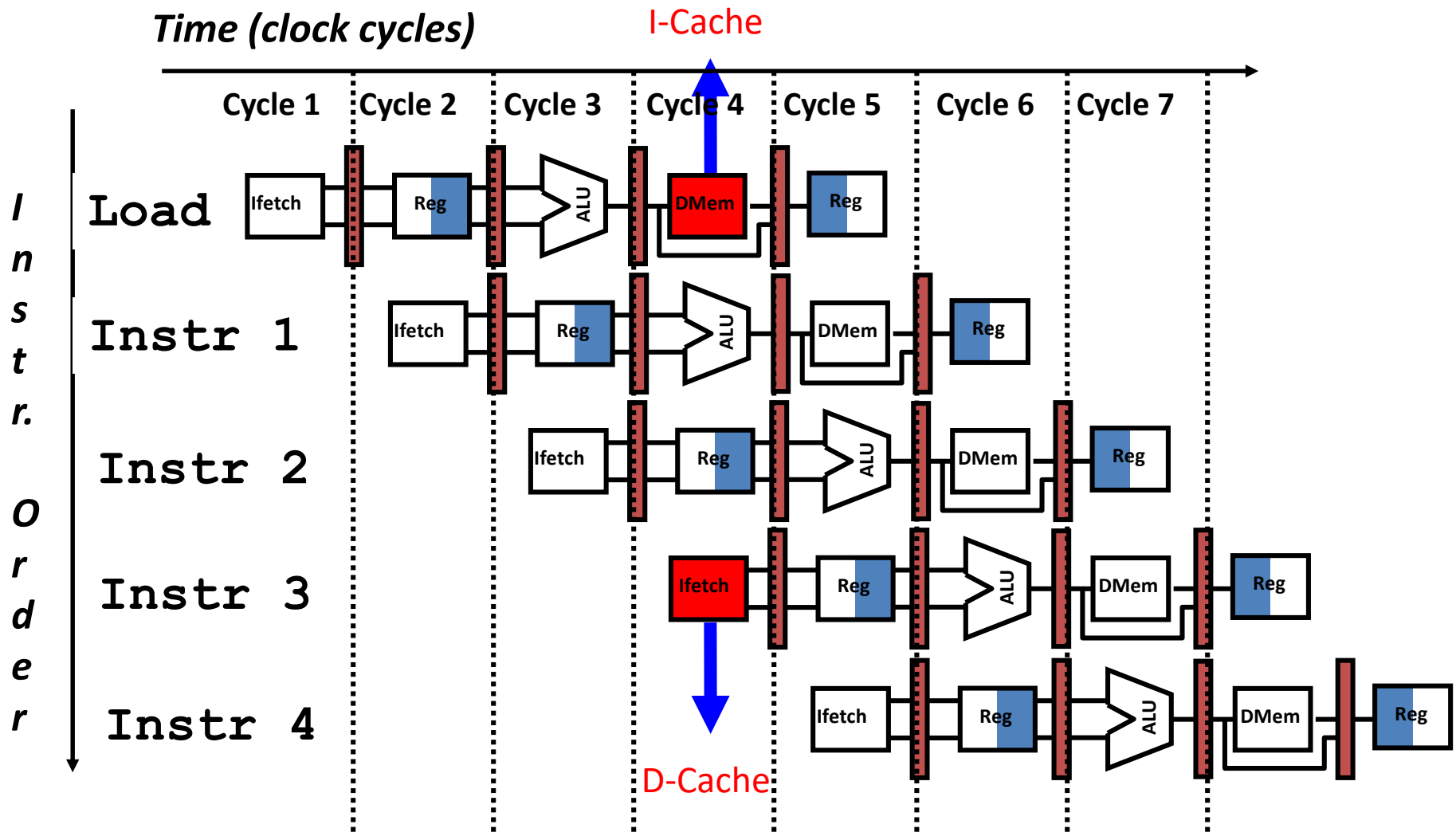
Review

- Structural hazards – Solution1: Stall



Review

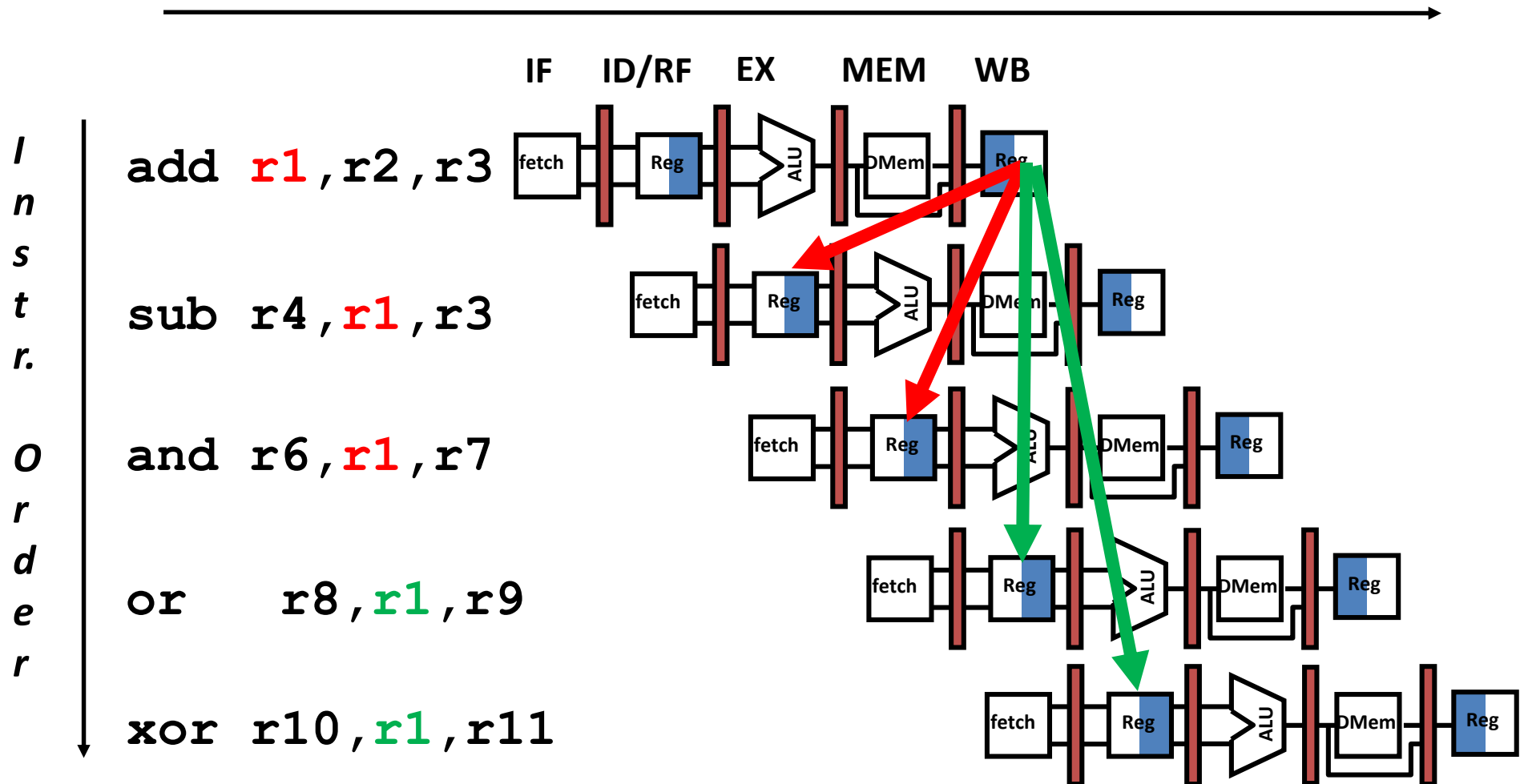
- Structural hazards – Solution 2: I-Cache and D-Cache



Review

- Data hazards:** Instruction depends on result of prior instruction still in the pipeline (missing sock)

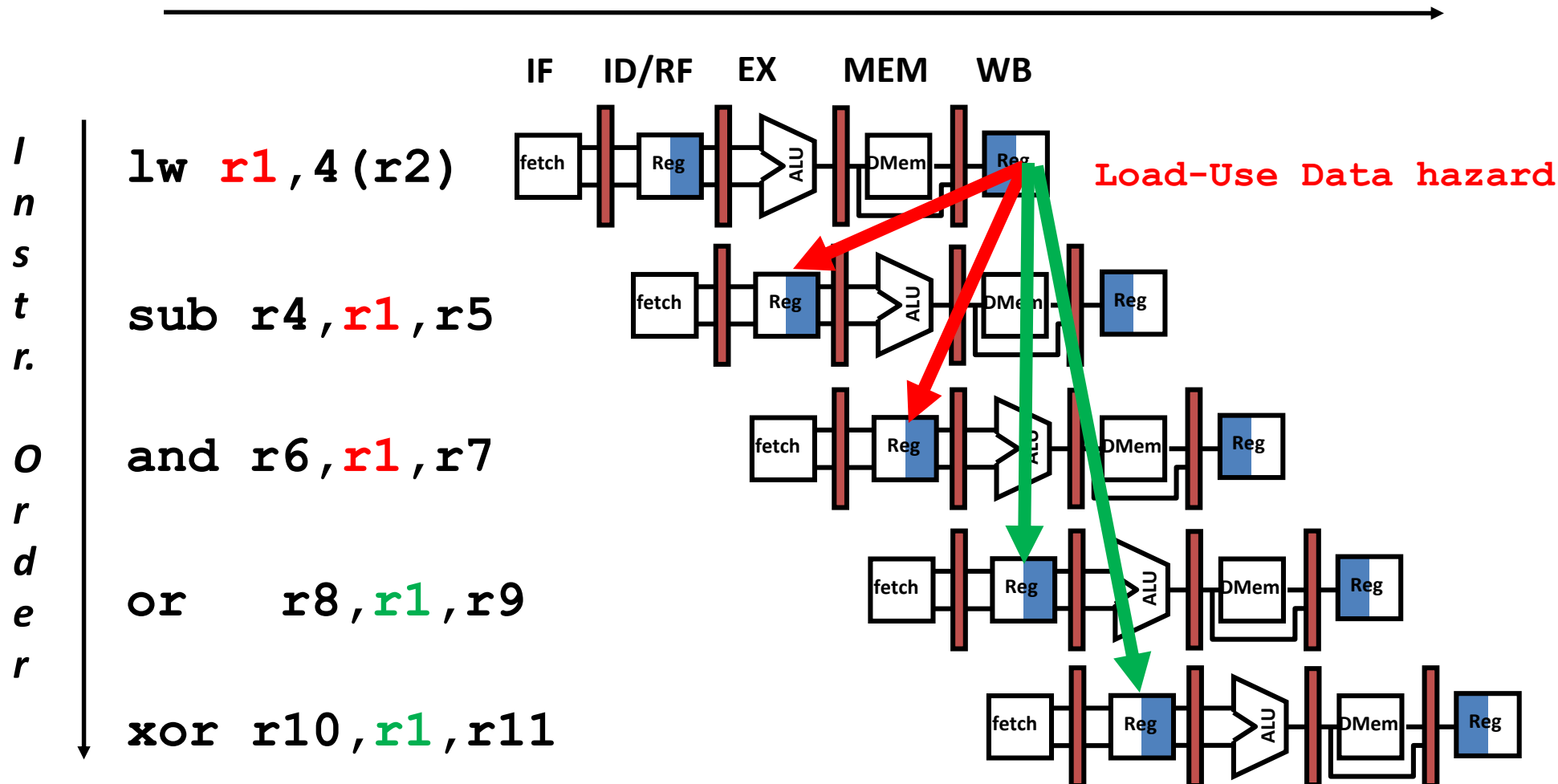
Time (clock cycles)



Review

- Data hazards:** Instruction depends on result of prior instruction still in the pipeline (missing sock)

Time (clock cycles)

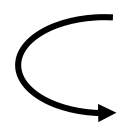


Review

Three Generic Data Hazards:

Read After Write (RAW)

Instr_j tries to read operand before Instr_i writes it

 I: add **r1**, r2, r3
J: sub r4, **r1**, r3

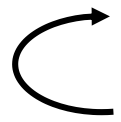
Caused by a “**Dependence**” (in compiler nomenclature). This hazard results from an actual need for communication.

Review

Three Generic Data Hazards:

Write After Read (WAR)

InstrJ writes operand **before** InstrI reads it



```
I:  sub  r4, r1, r3
J:  add  r1, r2, r3
K:  mul  r6, r1, r7
```

Called an “**anti-dependence**” by compiler writers.
This results from reuse of the name “**r1**”.

Can't happen in MIPS 5 stage pipeline because:

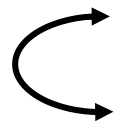
- All instructions take 5 stages, and
- Reads are always in stage 2, and
- Writes are always in stage 5

Review

Three Generic Data Hazards:

Write After Write (WAW)

InstrJ writes operand **before** InstrI writes it.



```
I:  sub  r1, r4, r3
J:  add  r1, r2, r3
K:  mul  r6, r1, r7
```

Called an “**output dependence**” by compiler writers

This also results from the reuse of name “**r1**”.

Can't happen in MIPS 5 stage pipeline because:

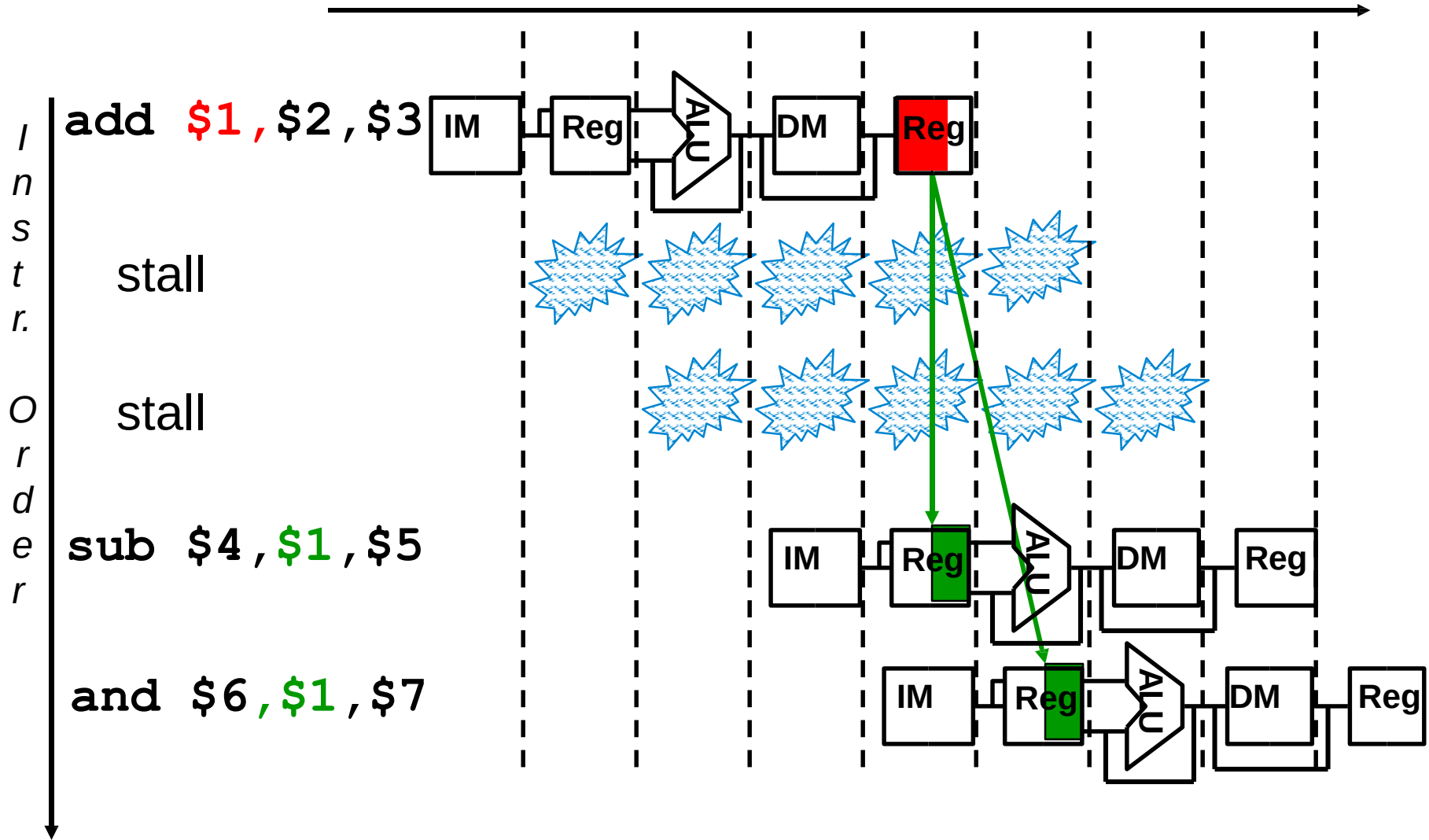
- All instructions take 5 stages, and
- Writes are always in stage 5

Will see WAR and WAW in later more complicated pipes

Review

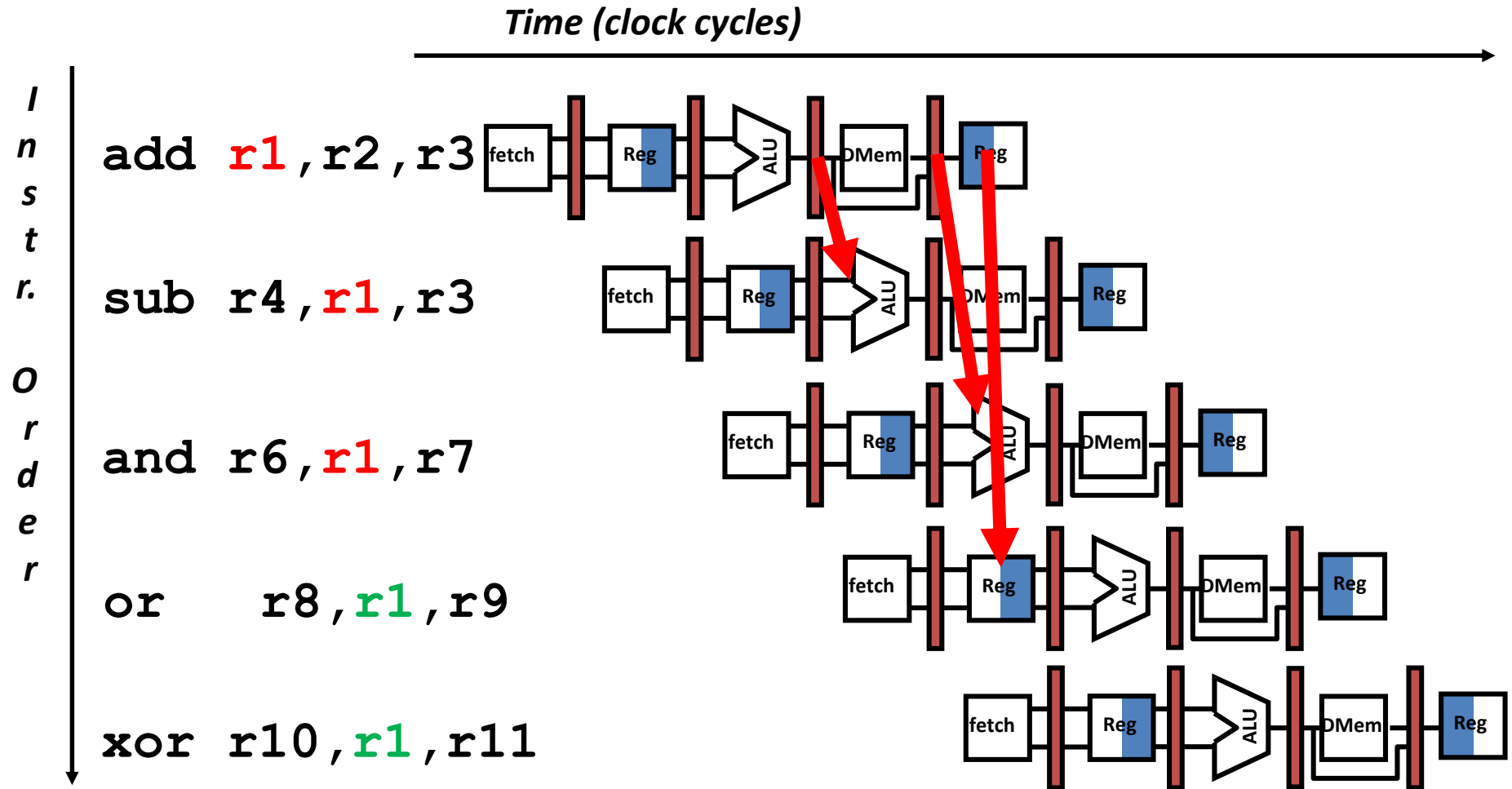
- **Data hazards – Solution 1:** Stall

Time (clock cycles)



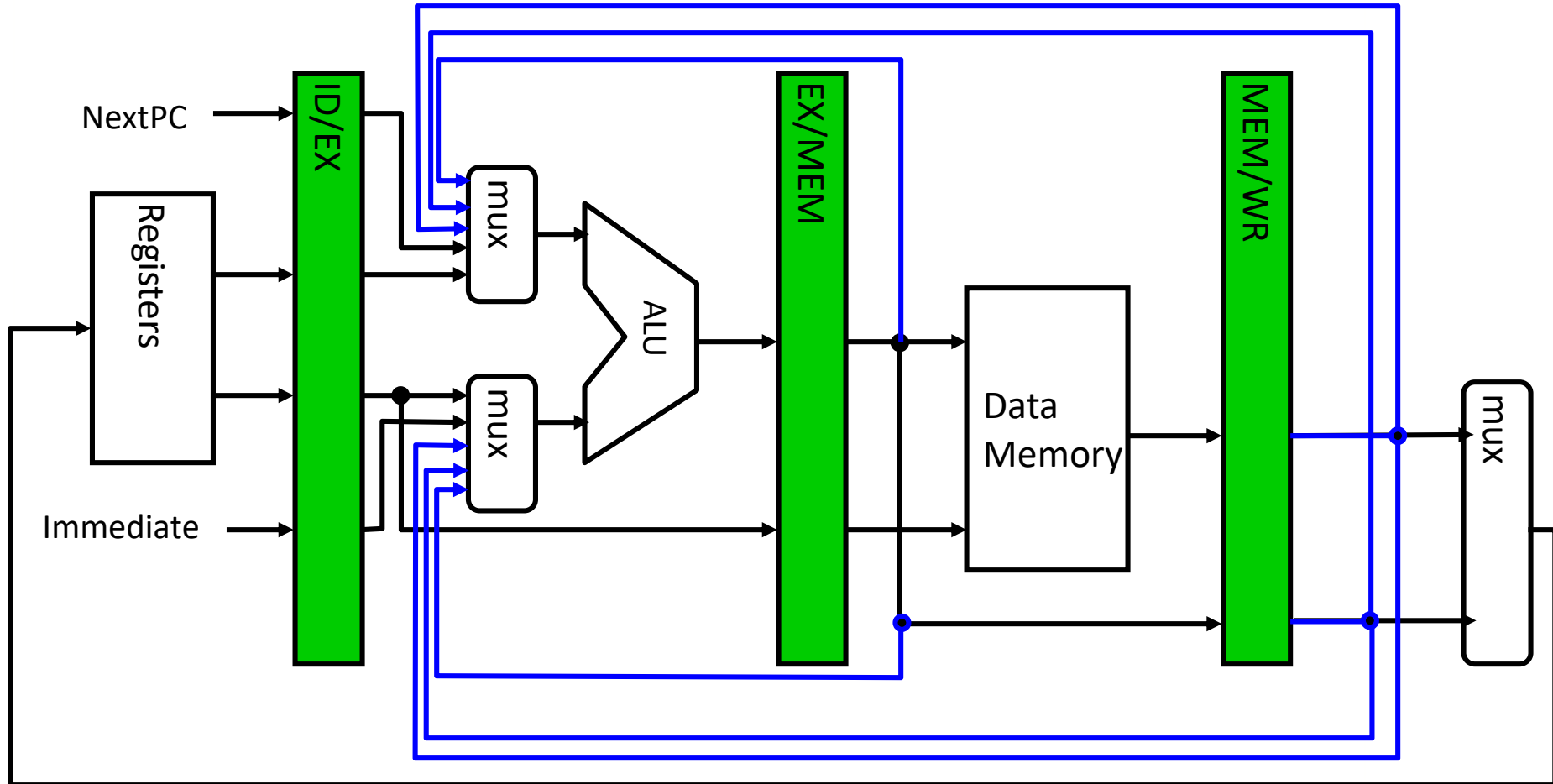
Review

- **Data hazards – Solution 2:** Forwarding



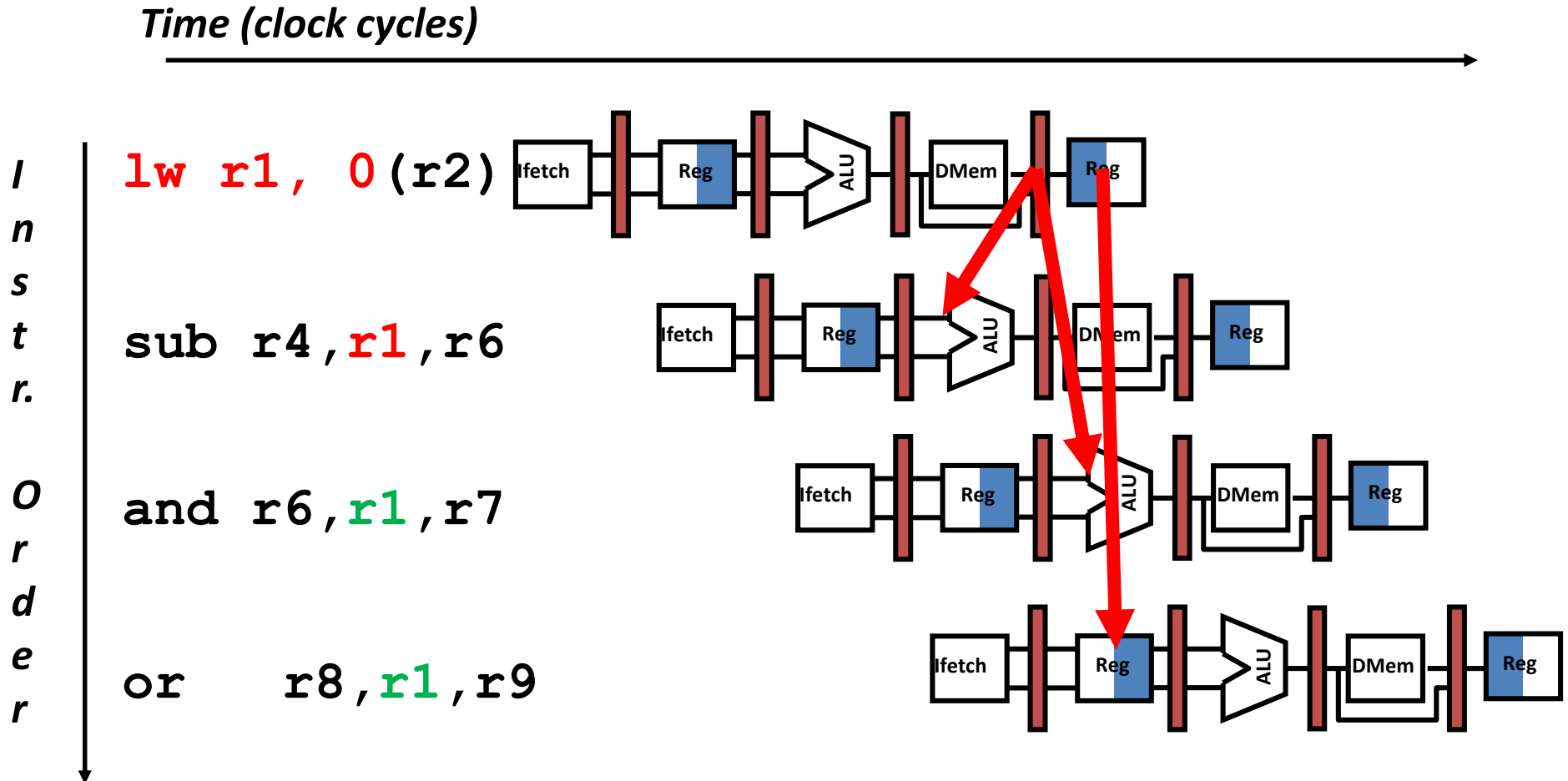
Review

- Hardware Change for Forwarding



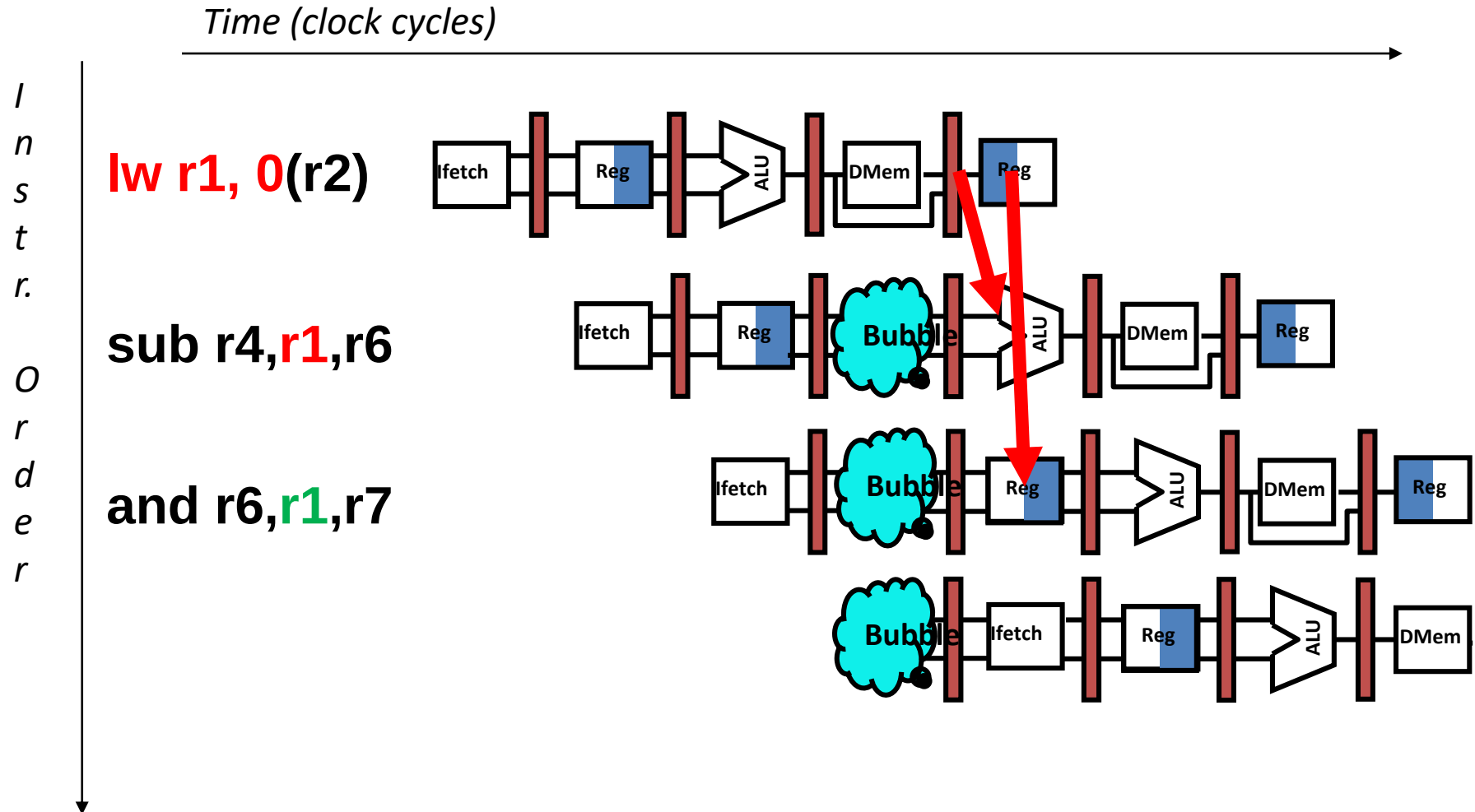
Review

- Data Hazard Even with Forwarding



Review

- Data Hazard Even with Forwarding – Solution: Stall



Review

- **Data Hazards (Load Hazards)– Solution:** Software Scheduling

Try producing fast code for

$a = b + c;$

$d = e - f;$

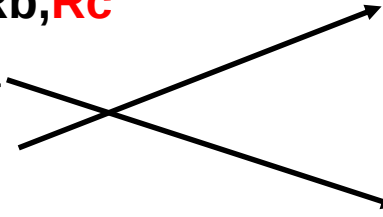
assuming a, b, c, d, e, and f in memory.

Slow code:

```
LW    Rb,b
LW    Rc,c
ADD   Ra,Rb,Rc
SW    a,Ra
LW    Re,e
LW    Rf,f
SUB   Rd,Re,Rf
SW    d,Rd
```

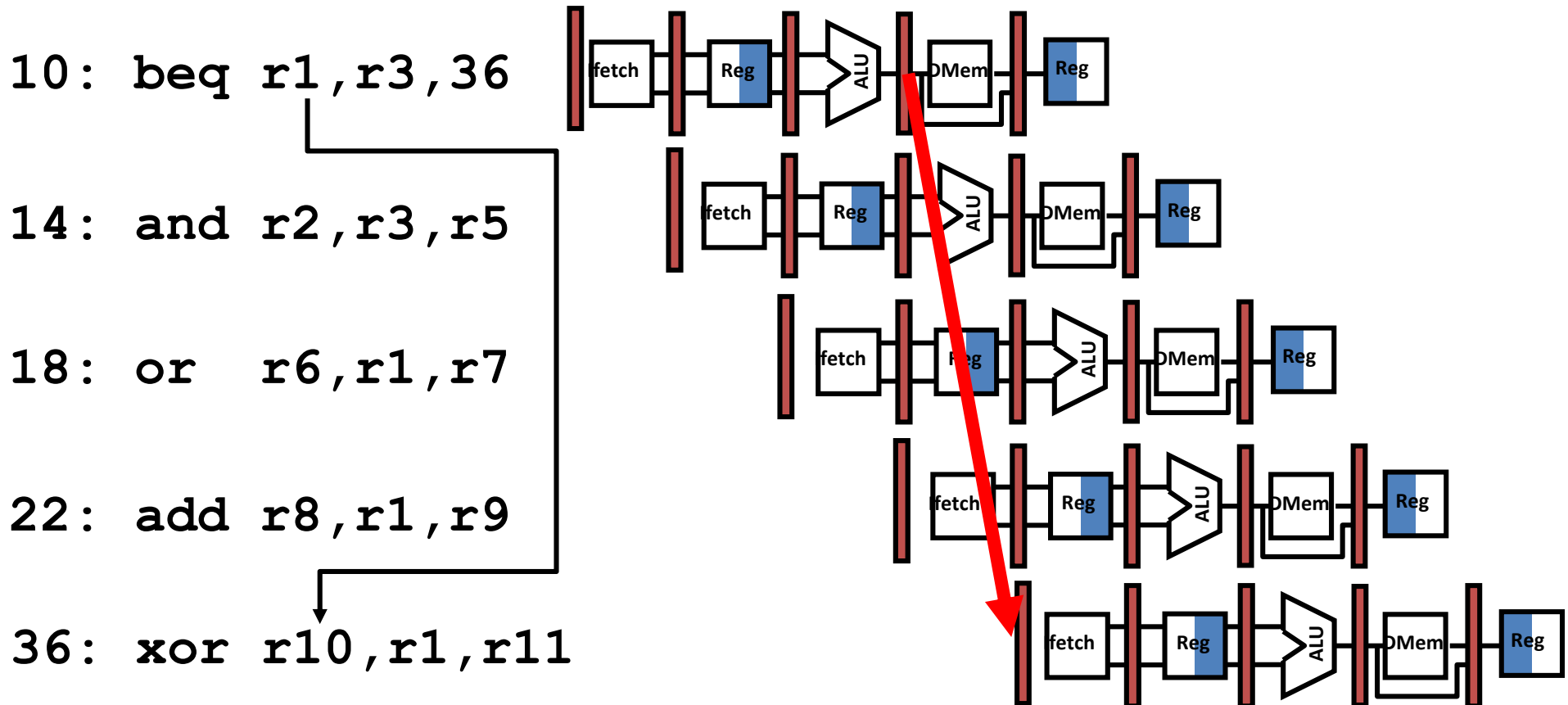
Fast code:

```
LW    Rb,b
LW    Rc,c
LW    Re,e
ADD   Ra,Rb,Rc
LW    Rf,f
SW    a,Ra
SUB   Rd,Re,Rf
SW    d,Rd
```



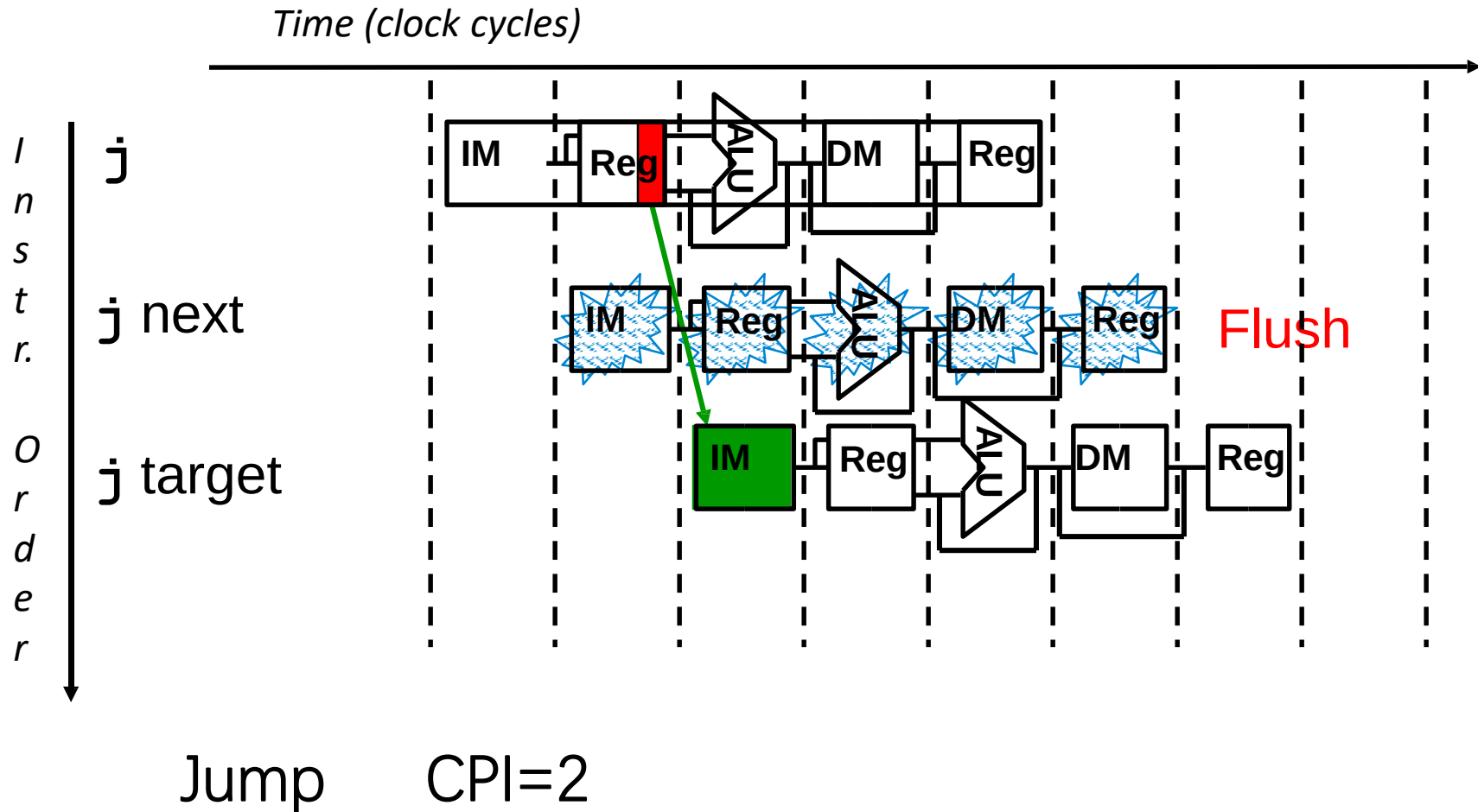
Review

- Control hazards:** Caused by delay between the fetching of instructions and decisions about changes in control flow (branches and jumps).



Review

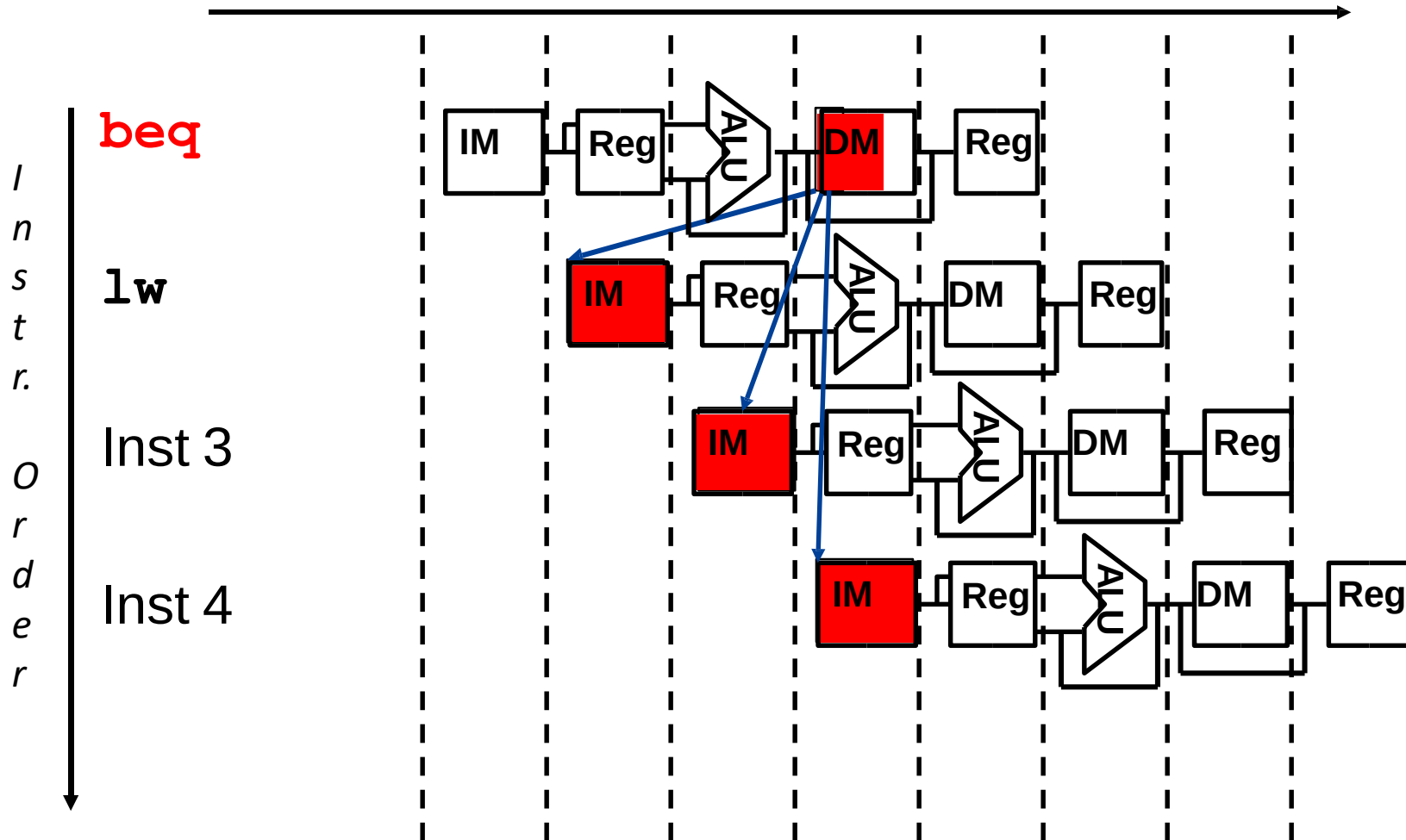
- Control hazards – Solution for J Instructions: Flush



Review

- Control hazards – Solution for Branch: Flush

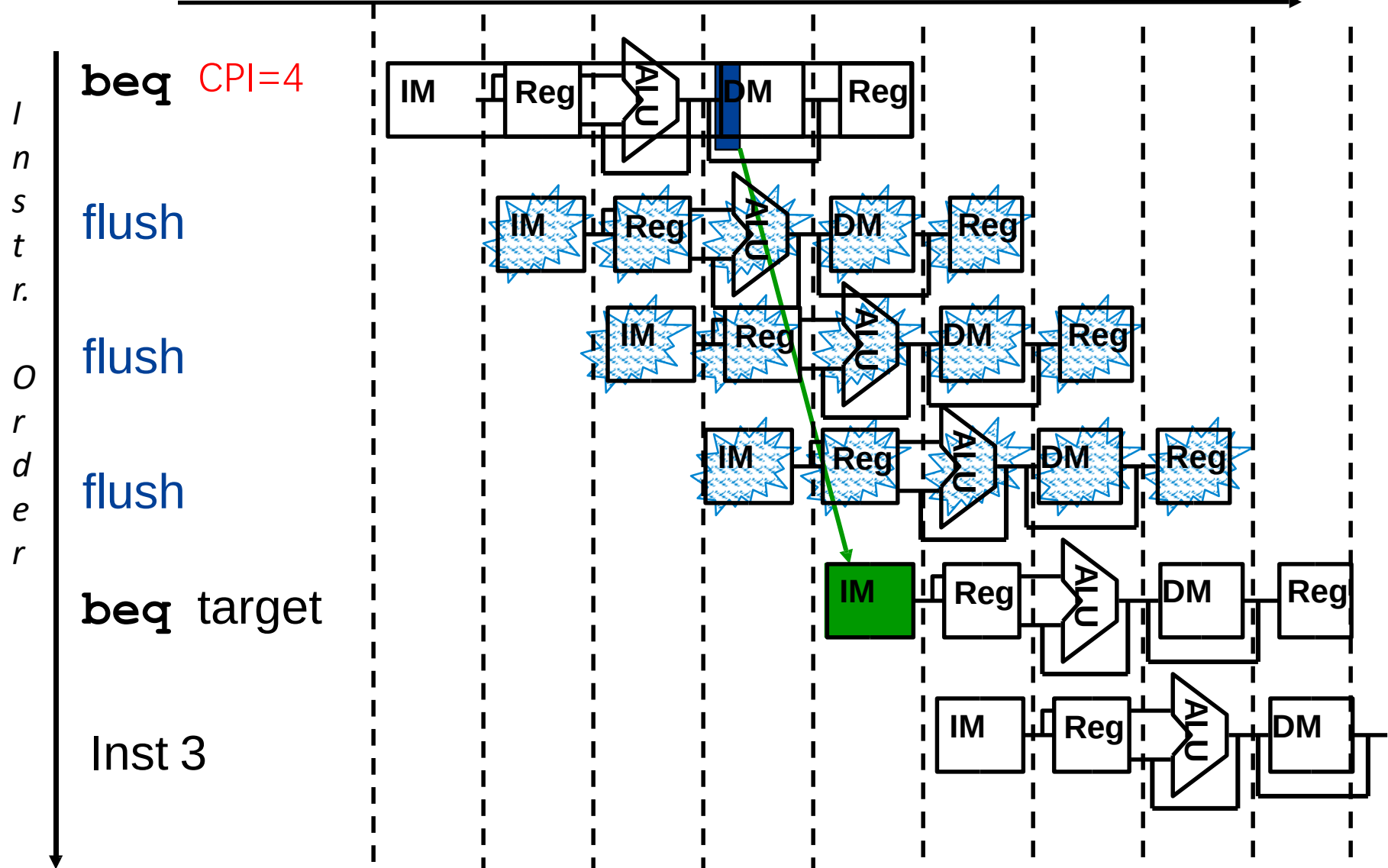
Time (clock cycles)



Review

- Control hazards – Solution for Branch: Flush

Time (clock cycles)



Review

- **Example: Branch Stall Impact**

If 30% branch, Stall 3 cycles significant

Two part solution:

Determine branch taken or not sooner, AND

Compute taken branch address earlier

MIPS branch tests if register = 0 or $\neq$ 0

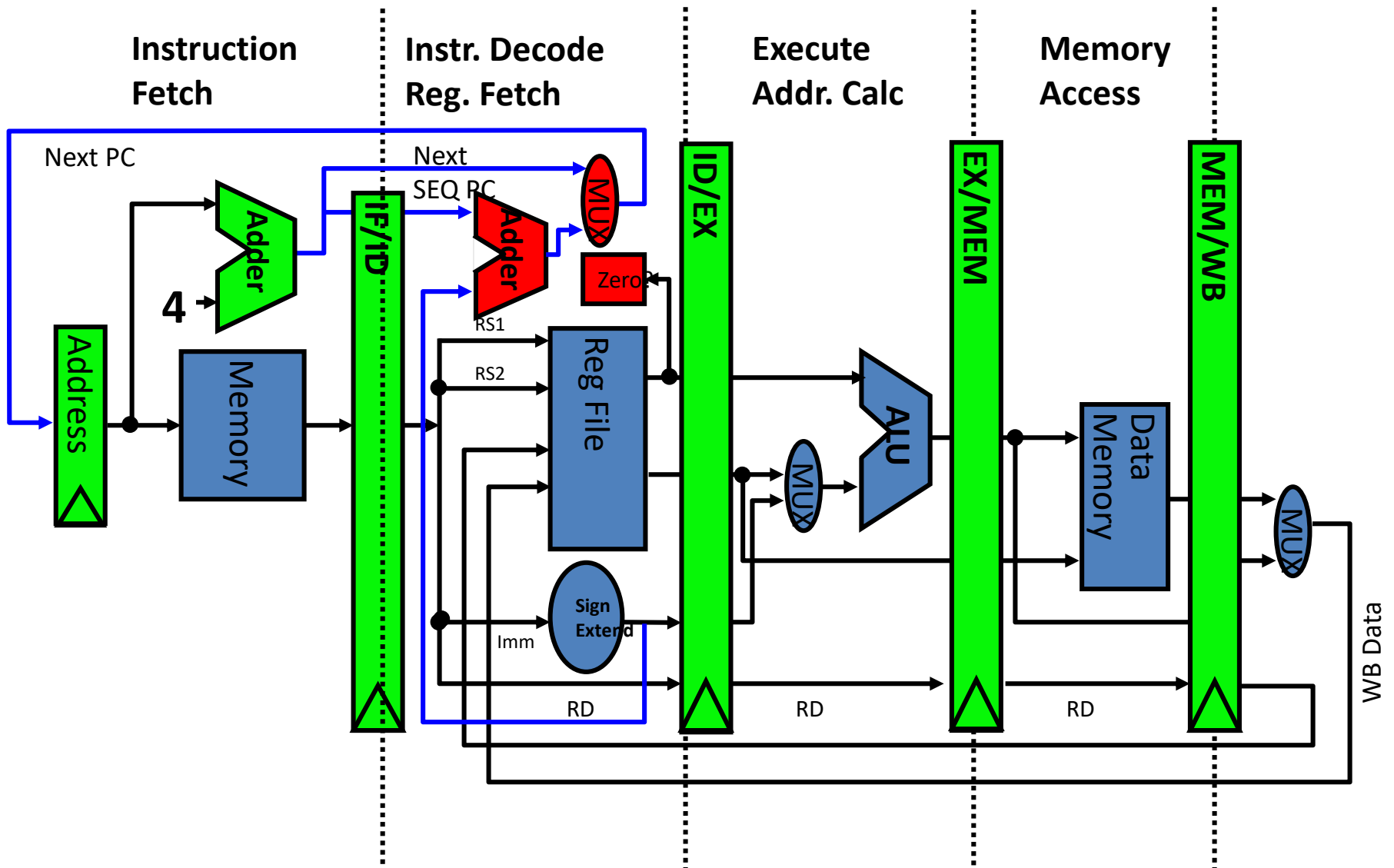
MIPS Solution:

Move Zero test to ID/RF stage

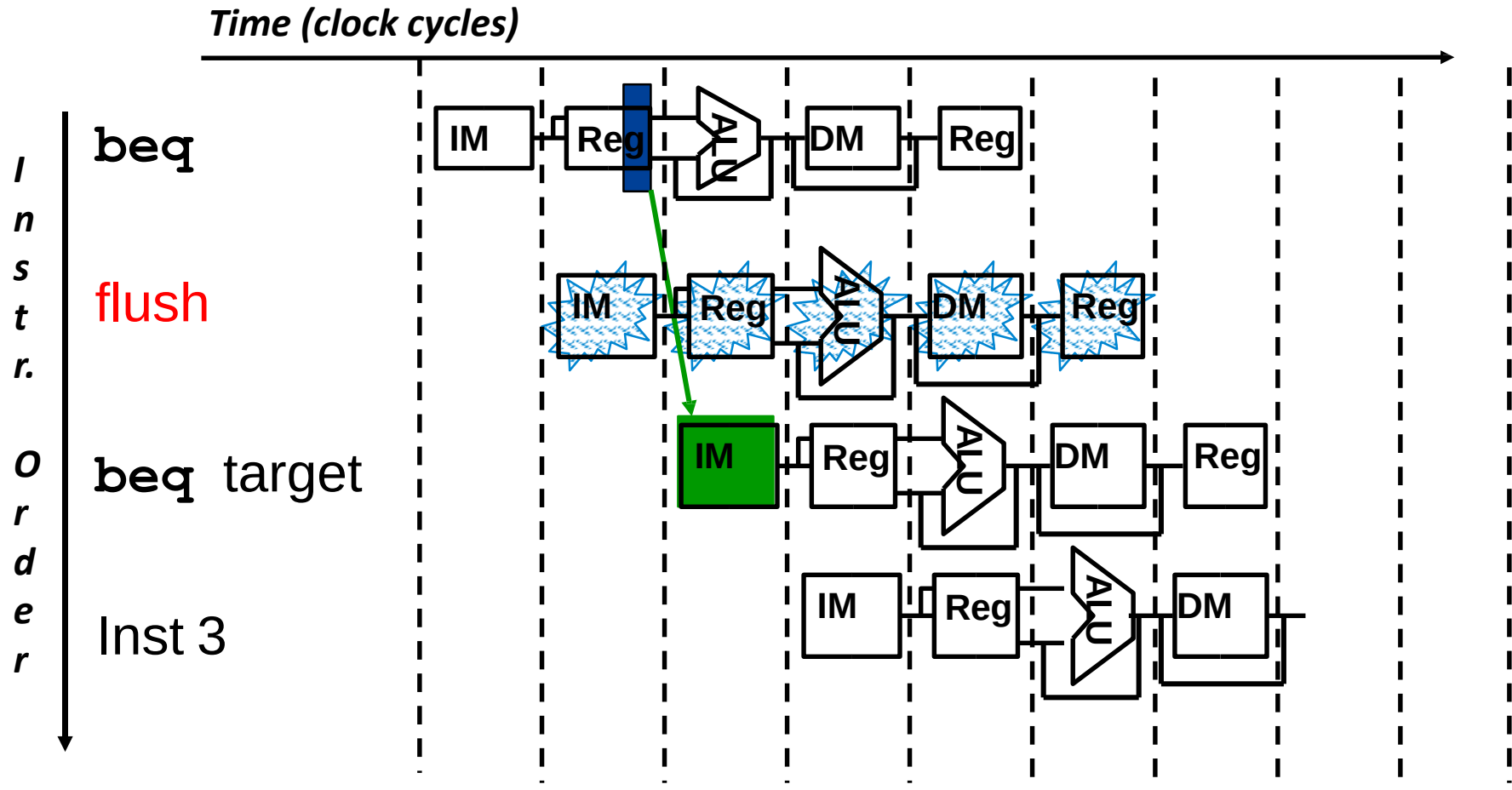
Adder to calculate new PC in ID/RF stage

1 clock cycle penalty for branch versus 3

Review



Review



Review

- **Four Branch Hazard Alternatives**

#1: Stall until branch direction is clear

#2: Predict Branch Not Taken

Execute successor instructions in sequence

“Squash” instructions in pipeline if branch actually taken

Advantage of late pipeline state update

47% MIPS branches not taken on average

PC+4 already calculated, so use it to get next instruction

#3: Predict Branch Taken

53% MIPS branches taken on average

But haven't calculated branch target address in MIPS

MIPS still incurs 1 cycle branch penalty

Other machines: branch target known before outcome

Review

- **Four Branch Hazard Alternatives**

#4: Delayed Branch

Define branch to take place **AFTER** a following instruction

```
branch instruction  
sequential successor1  
sequential successor2  
.....  
sequential successorn  
branch target if taken
```

1 slot delay allows proper decision and branch target address in 5 stage pipeline
MIPS uses this

Review

- **CPI (Cycles Per Instruction)**
- Pipeline CPI = Ideal pipeline CPI + Structural Stalls + Data Hazard Stalls + Control Stalls + Memory Stalls
 - Ideal pipeline CPI: measure of the maximum performance attainable by the implementation
 - Structural hazards: HW cannot support this combination of instructions
 - Data hazards: Instruction depends on result of prior instruction still in the pipeline
 - Control hazards: Caused by delay between the fetching of instructions and decisions about changes in control flow (branches and jumps)

ILP vs. Parallel Computers

Instruction-Level Parallelism (ILP)

Instructions of single process (or thread) executed in parallel
Parallel components must *appear* to execute in sequential program order

Parallel Computers or Multiprocessors

Program divided into multiple processes (or threads)
Instructions of multiple threads executed in parallel
Typically also involves ILP within each thread
No a priori sequential order between parallel threads

Dynamic Scheduling - Basics

The situation:

```
DIV.D F0, F2, F4
ADD.D F10, F0, F8
MULT.D F6, F6, F14
```

The problem:

ADD stalls due to RAW hazard

MULT stalls because ADD stalls

Example

	1	2	3	4	5	6	7	8	
DIV.D	IF	ID	E/	E/	E/	E/	MEM	WB	
ADD.D		IF	ID	**	**	**	E+	E+	
MULT.D			IF	**	**	**	ID	E*	why stall?

In-order execution limits performance

Dynamic Scheduling - Basics (Cont.)

Solutions

Static Scheduling & Dynamic Scheduling

Static Scheduling (Software) Compiler reorganizes instructions

+

+

(Will see more later)

Dynamic Scheduling (Hardware)

Hardware reorganizes instructions

+

+

+

Dynamic Scheduling - Basics (Cont.)**

Solutions

Static Scheduling Dynamic Scheduling

Static Scheduling (Software) Compiler reorganizes instructions

+ Simpler hardware

+

(Will see more later)

Dynamic Scheduling (Hardware)

Hardware reorganizes instructions

+

+

+

Dynamic Scheduling - Basics (Cont.)**

Solutions

Static Scheduling Dynamic Scheduling

Static Scheduling (Software) Compiler reorganizes instructions

- + Simpler hardware

- + Can use more powerful algorithms (Will see more later)

Dynamic Scheduling (Hardware)

Hardware reorganizes instructions

- +

- +

- +

Dynamic Scheduling - Basics (Cont.)**

Solutions

Static Scheduling Dynamic Scheduling

Static Scheduling (Software) Compiler reorganizes instructions

- + Simpler hardware

- + Can use more powerful algorithms (Will see more later)

Dynamic Scheduling (Hardware)

Hardware reorganizes instructions

- + Handles dependences unknown at compile time

- +

- +

Dynamic Scheduling - Basics (Cont.)**

Solutions

Static Scheduling Dynamic Scheduling

Static Scheduling (Software) Compiler reorganizes instructions

- + Simpler hardware
- + Can use more powerful algorithms (Will see more later)

Dynamic Scheduling (Hardware)

Hardware reorganizes instructions

- + Handles dependences unknown at compile time
- + It simplifies the compiler
- +

Dynamic Scheduling - Basics (Cont.)**

Solutions

Static Scheduling Dynamic Scheduling

Static Scheduling (Software) Compiler reorganizes instructions

- + Simpler hardware
- + Can use more powerful algorithms (Will see more later)

Dynamic Scheduling (Hardware)

Hardware reorganizes instructions

- + Handles dependences unknown at compile time
- + It simplifies the compiler
- + Software is more portable

Dynamic Scheduling - Basics (Cont.)

In-order execution - Static

- Instructions sent to execution units sequentially

- Stall instruction $i + 1$ if instruction i stalls for lack of operands

Out-of-order execution - Dynamic

- Send independent instructions to execution units as soon as possible

Dynamic Scheduling Basics (Cont.)

Original simple pipeline

ID – decode, check structural hazards and data hazards, read operands

EX – execute

Dynamic pipeline

Split ID (“**issue to execution unit**”) into two parts

Check for structural hazards

Wait for data dependences

New organization (conceptual):

Issue – decode, check structural hazards, read ready operands

ReadOps – wait until data hazards clear, read operands, begin execution

Issue stays in-order; ReadOps/beginning of EX is out-of-order

Dynamic Scheduling Basics (Cont.)

Dynamic scheduling can create WAW, WAR hazards, and imprecise exceptions

WAW hazards with dynamic scheduling

```
DIV.D    F0, F2, F4
ADD.D    F10, F0, F8
MUL.D    F10, F8, F14
```

WAR hazards with dynamic scheduling

```
DIV.D    F0, F2, F4
ADD.D    F10, F0, F8
MUL.D    F8, F8, F14
```

Can always stall,

but more aggressive solution with ***register renaming***

Scoreboarding

It is named after the CDC 6600 scoreboard, which developed this capability.

A technique for allowing instructions to execute out of order

- Sufficient resources and no data dependences
- Take responsibility for instruction issue and execution, including all hazard detection
- Solve the WAR and WAW hazards

Goal:

- Maintain an execution rate of one instruction per clock cycle (when there are no structural hazards) by executing an instruction as early as possible

Basic rules:

- Every instruction goes through the scoreboard, where a record of the data dependences is constructed
- The scoreboard then determines when the instruction can read its operands and begin execution

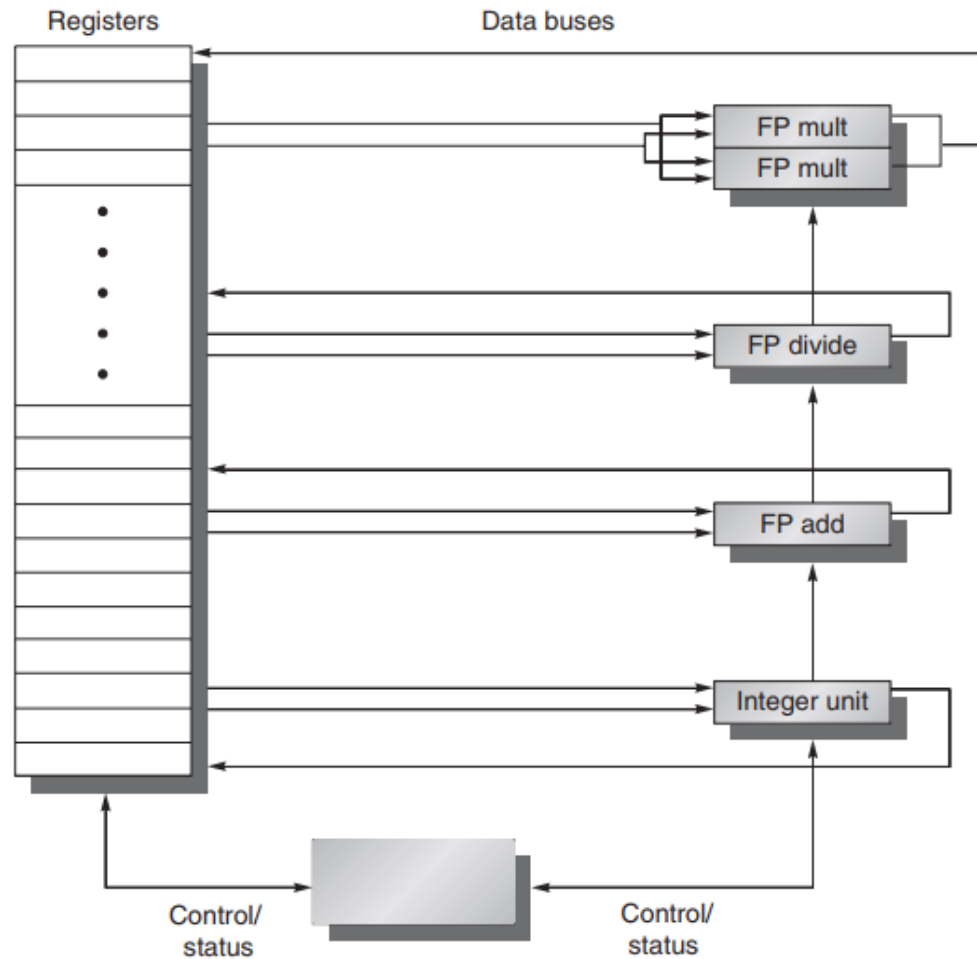
Some History - CDC 6600

The flagship of the 6000 series of mainframe computer systems

- Delivered in 1965, considered to be the first successful supercomputer.
- The Scoreboarding tech was developed by Seymour Cray for CDC 6600 in 1964.



The Basic Structure of Scoreboard



Four Stages of Scoreboarding Tech.

1. **Issue**—get instruction from FP Op Queue
If a functional unit for the instruction is free and no other active instruction has the same destination register & issue the instruction.
2. **Read operands** —monitor the availability of the source operands
If no earlier issued active instruction is going to write a source operand, it is available & the functional unit to proceed to read the operands from the registers and begin execution.
3. **Execute**—operate on operands (EX)
The functional unit begins execution upon receiving operands. The result is ready, it notifies the scoreboard that it has completed execution.
4. **Write result**—finish execution (WB)
Once the scoreboard is aware that the functional unit has completed execution, the scoreboard checks for WAR hazards and stalls the completing instruction, if necessary.

Three Parts of Scoreboard

1. **Instruction status**—Indicates which of the four steps the instruction is in
2. **Functional unit status**—Indicates the state of the functional unit (FU)
 - Busy:** Indicates whether the FU is busy or not
 - Op:** Operation to perform in the unit (e.g., + or −)
 - Fi:** Destination register
 - Vj, Vk:** Value of Source operands/Source-register numbers
 - Qj, Qk:** FUs producing source registers (value to be written)
 - Rj, Rk:** Flags indicating when Fj, Fk are ready and not yet read
 - Note: Set to No after operands are read
3. **Register result status**—Indicates which functional unit will write each register, if an active instruction has the register as its destination.
 - Blank when no pending instructions that will write that register.

Three Parts of Scoreboard

Instruction		Instruction status			
		Issue	Read operands	Execution complete	Write result
L.D	F6,34(R2)	√	√	√	√
L.D	F2,45(R3)	√	√	√	
MUL.D	F0,F2,F4	√			
SUB.D	F8,F6,F2	√			
DIV.D	F10,F0,F6	√			
ADD.D	F6,F8,F2				

Functional unit status									
Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
Integer	Yes	Load	F2	R3				No	
Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
Mult2	No								
Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status									
	F0	F2	F4	F6	F8	F10	F12	...	F30
FU	Mult1	Integer			Add	Divide			

Scoreboard Example

Example:

LD F6, 34 (R2)

LD F2, 45 (R3)

MULTD F0, F2, F4

SUBD F8, F6, F2

DIVD F10, F0, F6

ADDD F6, F8, F2

Example speed:

- 2 cycles for fl +, - ;
- 10 cycles for *;
- 40 cycles for /;

<i>Instruction status:</i>				<i>Read</i>	<i>Exec</i>	<i>Write</i>	
Instruction		<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Oper</i>	<i>Comp</i>	<i>Result</i>
LD	F6	34+	R2				
LD	F2	45+	R3				
MULTD	F0	F2	F4				
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

[illegible]

<i>Functional unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

No
No
No
No
No

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
<i>FU</i>									

FU	
------	--

Scoreboard Example: Cycle 1

Instruction status:

Instruction		<i>j</i>	<i>k</i>	Issue	Read Oper	Exec Comp	Write Result
LD	F6	34+	R2	1			
LD	F2	45+	R3				
MULTD	F0	F2	F4				
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

Functional unit status:

Time	Name	Busy	Op	dest	<i>S1</i> <i>Fj</i>	<i>S2</i> <i>Fk</i>	<i>FU</i> <i>Qj</i>	<i>FU</i> <i>Qk</i>	<i>Fj?</i> <i>Rj</i>	<i>Fk?</i> <i>Rk</i>
	Integer	Yes	Load	F6		R2				Yes
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

Register result status:

Clock
1

	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
<i>FU</i>				Integer					

Instruction		j	k
LD	F6	34+	R2
LD	F2	45+	R3
MULTD	F0	F2	F4
SUBD	F8	F6	F2
DIVD	F10	F0	F6
ADDD	F6	F8	F2

Instruction status:

<i>Instruction status:</i>				Read	Exec	Write
Instruction	<i>j</i>	<i>k</i>	Issue	Oper	Comp	Result
LD	F6	34+	R2	1	2	
LD	F2	45+	R3			
MULTD	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

- Issue 2nd LD?

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	Yes	Load	F6		R2				Yes
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
2	FU	Integer								

Instruction status:

<i>Instruction status:</i>				Read	Exec	Write
Instruction	<i>j</i>	<i>k</i>	Issue	Open	Comp	Result
LD	F6	34+	R2	1	2	3
LD	F2	45+	R3			
MULTD	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	Yes	Load	F6		R2				No
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
3	FU	Integer								

Instruction status:

<i>Instruction status:</i>				<i>Read</i>	<i>Exec</i>	<i>Write</i>
Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Open</i>	<i>Comp</i>	<i>Result</i>
LD	F6	34+	R2	1	2	3
LD	F2	45+	R3			
MULTD	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

- Issue MULT?

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	Yes	Load	F6		R2				No
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
3	FU	Integer								

<i>Instruction status:</i>				<i>Read</i>	<i>Exec</i>	<i>Write</i>
Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Oper</i>	<i>Comp</i>	<i>Result</i>
LD	F6	34+	R2	1	2	3
LD	F2	45+	R3			
MULTD	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

Write
to Result
4

Functional unit status:

			<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>	
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

<i>Busy</i>	<i>Op</i>	<i>Pi</i>	<i>Pj</i>	<i>Pk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
No								
No								

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
4	FU	Integer								

FU	Integer
------	---------

Instruction status:

<i>Instruction status:</i>				<i>Read</i>	<i>Exec</i>	<i>Write</i>	
Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Oper</i>	<i>Comp</i>	<i>Result</i>	
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5			
MULTD	F0	F2	F4				
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

Functional unit status:

<i>l unit status:</i>		<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>		
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>F1</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	Yes	Load	F2		R3				Yes
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
5	FU	Integer								

Scoreboard Example: Cycle 6

Instruction status:

<i>Instruction status:</i>				<i>Read</i>	<i>Exec</i>	<i>Write</i>	
Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Oper</i>	<i>Comp</i>	<i>Result</i>	
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6		
MULTD	F0	F2	F4	6			
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

Functional unit status:

<i>Unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	Yes	Load	F2		R3				Yes
	Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
	Mult2	No								
	Add	No								
	Divide	No								

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
6	FU	Mult1	Integer							

Instruction status:

<i>Instruction status:</i>				<i>Read</i>	<i>Exec</i>	<i>Write</i>	
Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Oper</i>	<i>Comp</i>	<i>Result</i>	
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	
MULTD	F0	F2	F4	6			
SUBD	F8	F6	F2	7			
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

Functional unit status:

<i>l unit status:</i>			<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>	
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	Yes	Load	F2		R3				No
	Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
	Mult2	No								
	Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
	Divide	No								

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
7	FU	Mult1	Integer			Add				

Scoreboard Example: Cycle 7

Instruction status:

Instruction		<i>j</i>	<i>k</i>	Issue	Read Oper	Exec Comp	Write Result
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	
MULTD	F0	F2	F4	6			
SUBD	F8	F6	F2	7			
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

• Read multiply operands?

Functional unit status:

Time	Name	Busy	Op	dest <i>Fi</i>	<i>S1</i> <i>Fj</i>	<i>S2</i> <i>Fk</i>	<i>FU</i> <i>Qj</i>	<i>FU</i> <i>Qk</i>	<i>Fj?</i> <i>Rj</i>	<i>Fk?</i> <i>Rk</i>
	Integer	Yes	Load	F2		R3				No
	Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
	Mult2	No								
	Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
	Divide	No								

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
7	<i>FU</i>	Mult1	Integer			Add				

Scoreboard Example: Cycle 8a

Instruction status:

Instruction		<i>j</i>	<i>k</i>	Issue	Read Oper	Exec Comp	Write Result
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	
MULTD	F0	F2	F4	6			
SUBD	F8	F6	F2	7			
DIVD	F10	F0	F6	8			
ADDD	F6	F8	F2				

- Read operands for MULT & SUB?

Functional unit status:

Time	Name	Busy	Op	dest <i>Fi</i>	<i>S1</i> <i>Fj</i>	<i>S2</i> <i>Fk</i>	<i>FU</i> <i>Qj</i>	<i>FU</i> <i>Qk</i>	<i>Fj?</i> <i>Rj</i>	<i>Fk?</i> <i>Rk</i>
	Integer	Yes	Load	F2		R3				No
	Mult1	Yes	Mult	F0	F2	F4	Integer		No	Yes
	Mult2	No								
	Add	Yes	Sub	F8	F6	F2		Integer	Yes	No
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
8	<i>FU</i>	Mult1	Integer			Add	Divide			

Scoreboard Example: Cycle 8b

Instruction status:

				Read	Exec	Write
Instruction	<i>j</i>	<i>k</i>	Issue	Oper	Comp	Result
LD	F6	34+ R2	1	2	3	4
LD	F2	45+ R3	5	6	7	8
MULTD	F0	F2 F4	6			
SUBD	F8	F6 F2	7			
DIVD	F10	F0 F6	8			
ADDD	F6	F8 F2				

Functional unit status:

l unit status:

				dest	S1	S2	FU	FU	Fj?	Fk?
Time	Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
	Integer	No								
	Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
	Mult2	No								
	Add	Yes	Sub	F8	F6	F2			Yes	Yes
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
8	FU Mult1				Add	Divide			

Scoreboard Example: Cycle 9

Instruction status:

Instruction status:

Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Read Oper</i>	<i>Exec Comp</i>	<i>Write Result</i>	
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9		
SUBD	F8	F6	F2	7	9		
DIVD	F10	F0	F6	8			
ADDD	F6	F8	F2				

Functional unit status:

Note → Remaining

Time	Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
	Integer	No								
10	Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
	Mult2	No								
2	Add	Yes	Sub	F8	F6	F2			Yes	Yes
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Note →
Remaining

Register result status:

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
9	FU	Mult1				Add	Divide			

Scoreboard Example: Cycle 9

Instruction status:

Instruction		j	k	Issue	Read Oper	Exec Comp	Write Result
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9		
SUBD	F8	F6	F2	7	9		
DIVD	F10	F0	F6	8			
ADDD	F6	F8	F2				

• Issue ADDD?

Functional unit status:

Time	Name	Busy	Op	dest Fi	S1 Fj	S2 Fk	FU Qj	FU Qk	Fj? Rj	Fk? Rk
	Integer	No								
10	Mult1	Yes	Mult	F0	F2	F4			Yes	Yes
	Mult2	No								
2	Add	Yes	Sub	F8	F6	F2			Yes	Yes
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Note →
Remaining

Register result status:

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
9	FU	Mult1				Add	Divide			

Scoreboard Example: Cycle 10

Instruction status:

				Read	Exec	Write
Instruction		<i>j</i>	<i>k</i>	Issue	Oper	Comp Result
LD	F6	34+	R2	1	2	3 4
LD	F2	45+	R3	5	6	7 8
MULTD	F0	F2	F4	6	9	
SUBD	F8	F6	F2	7	9	
DIVD	F10	F0	F6	8		
ADDD	F6	F8	F2			

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
9	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
1	Add	Yes	Sub	F8	F6	F2			No	No
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
10	<i>FU</i>	Mult1				Add	Divide			

Scoreboard Example: Cycle 11

Instruction status:

				Read	Exec	Write
Instruction	<i>j</i>	<i>k</i>	Issue	Oper	Comp	Result
LD	F6	34+ R2	1	2	3	4
LD	F2	45+ R3	5	6	7	8
MULTD	F0	F2 F4	6	9		
SUBD	F8	F6 F2	7	9	11	
DIVD	F10	F0 F6	8			
ADDD	F6	F8 F2				

Functional unit status:

<i>l unit status:</i>			<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>	
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
8	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
0	Add	Yes	Sub	F8	F6	F2			No	No
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
11	FU	Mult1				Add	Divide			

Scoreboard Example: Cycle 12

Instruction status:

				Read	Exec	Write
Instruction		<i>j</i>	<i>k</i>	Issue	Oper	Comp Result
LD	F6	34+	R2	1	2	3 4
LD	F2	45+	R3	5	6	7 8
MULTD	F0	F2	F4	6	9	
SUBD	F8	F6	F2	7	9	11 12
DIVD	F10	F0	F6	8		
ADDD	F6	F8	F2			

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
7	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
	Add	No								
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
12	<i>FU</i>	Mult1					Divide			

Scoreboard Example: Cycle 12

Instruction status:

Instruction		j	k	Issue	Read Oper	Exec Comp	Write Result
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9		
SUBD	F8	F6	F2	7	9	11	12
DIVD	F10	F0	F6	8			
ADDD	F6	F8	F2				

- Read operands for DIVD?
- Issue ADDD?

Functional unit status:

Time	Name	Busy	Op	dest Fi	S1 Fj	S2 Fk	FU Qj	FU Qk	Fj? Rj	Fk? Rk
	Integer	No								
7	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
	Add	No								
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
12	FU	Mult1					Divide			

Scoreboard Example: Cycle 13

Instruction status:

				Read	Exec	Write
Instruction	<i>j</i>	<i>k</i>	Issue	Oper	Comp	Result
LD	F6	34+ R2	1	2	3	4
LD	F2	45+ R3	5	6	7	8
MULTD	F0	F2 F4	6	9		
SUBD	F8	F6 F2	7	9	11	12
DIVD	F10	F0 F6	8			
ADDD	F6	F8 F2	13			

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
6	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
	Add	Yes	Add	F6	F8	F2			Yes	Yes
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
13	<i>FU</i>	Mult1			Add		Divide			

Scoreboard Example: Cycle 14

Instruction status:

				Read	Exec	Write
Instruction	j	k	Issue	Oper	Comp	Result
LD	F6	34+ R2	1	2	3	4
LD	F2	45+ R3	5	6	7	8
MULTD	F0	F2 F4	6	9		
SUBD	F8	F6 F2	7	9	11	12
DIVD	F10	F0 F6	8			
ADDD	F6	F8 F2	13	14		

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
5	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
2	Add	Yes	Add	F6	F8	F2			Yes	Yes
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
14	FU	Mult1			Add		Divide			

Scoreboard Example: Cycle 15

Instruction status:

				Read	Exec	Write
Instruction		<i>j</i>	<i>k</i>	Issue	Oper	Comp Result
LD	F6	34+	R2	1	2	3 4
LD	F2	45+	R3	5	6	7 8
MULTD	F0	F2	F4	6	9	
SUBD	F8	F6	F2	7	9	11 12
DIVD	F10	F0	F6	8		
ADDD	F6	F8	F2	13	14	

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
4	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
1	Add	Yes	Add	F6	F8	F2			No	No
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
15	<i>FU</i>	Mult1			Add		Divide			

Scoreboard Example: Cycle 16

Instruction status:

Instruction status:

Instruction	<i>j</i>	<i>k</i>	Issue	Read Oper	Exec Comp	Write Result	
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9		
SUBD	F8	F6	F2	7	9	11	12
DIVD	F10	F0	F6	8			
ADDD	F6	F8	F2	13	14	16	

Functional unit status:

<i>l unit status:</i>			<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>	
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
3	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
0	Add	Yes	Add	F6	F8	F2			No	No
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
16	<i>FU</i>	Mult1			Add		Divide			

Scoreboard Example: Cycle 17

Instruction status:

Instruction		j	k	Issue	Read Oper	Exec Comp	Write Result
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9		
SUBD	F8	F6	F2	7	9	11	12
DIVD	F10	F0	F6	8			
ADDD	F6	F8	F2	13	14	16	

- Why not write result of ADDD?

Functional unit status:

Time	Name	Busy	Op	dest Fi	S1 Fj	S2 Fk	FU Qj	FU Qk	Fj? Rj	Fk? Rk
	Integer	No								
2	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
	Add	Yes	Add	F6	F8	F2			No	No
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
17	FU	Mult1			Add		Divide			

Scoreboard Example: Cycle 17

Instruction status:

Instruction	j	k	Issue	Read Oper	Exec Comp	Write Result
LD	F6	34+ R2	1	2	3	4
LD	F2	45+ R3	5	6	7	8
MULTD	F0	F2 F4	6	9		
SUBD	F8	F6 F2	7	9	11	12
DIVD	F10	F0 F6	8			
ADDD	F6	F8 F2	13	14	16	

- Why not write result of ADDD?

WAR Hazard!

Functional unit status:

Time	Name	Busy	Op	dest Fi	S1 Fj	S2 Fk	FU Qj	FU Qk	Fj? Rj	Fk? Rk
	Integer	No								
2	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
	Add	Yes	Add	F6	F8	F2			No	No
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
17	FU Mult1			Add		Divide			

Scoreboard Example: Cycle 18

Instruction status:

				Read	Exec	Write
Instruction	<i>j</i>	<i>k</i>	Issue	Oper	Comp	Result
LD	F6	34+ R2	1	2	3	4
LD	F2	45+ R3	5	6	7	8
MULTD	F0	F2 F4	6	9		
SUBD	F8	F6 F2	7	9	11	12
DIVD	F10	F0 F6	8			
ADDD	F6	F8 F2	13	14	16	

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
1	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
	Add	Yes	Add	F6	F8	F2			No	No
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
18	FU	Mult1			Add		Divide			

Scoreboard Example: Cycle 19

Instruction status:

				Read	Exec	Write
Instruction		j	k	Issue	Oper	Comp Result
LD	F6	34+	R2	1	2	3 4
LD	F2	45+	R3	5	6	7 8
MULTD	F0	F2	F4	6	9	19
SUBD	F8	F6	F2	7	9	11 12
DIVD	F10	F0	F6	8		
ADDD	F6	F8	F2	13	14	16

Functional unit status:

<i>l unit status:</i>			<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>	
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
0	Mult1	Yes	Mult	F0	F2	F4			No	No
	Mult2	No								
	Add	Yes	Add	F6	F8	F2			No	No
	Divide	Yes	Div	F10	F0	F6	Mult1		No	Yes

Register result status:

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
19	FU	Mult1			Add		Divide			

Scoreboard Example: Cycle 20

Instruction status:

				Read	Exec	Write
Instruction		<i>j</i>	<i>k</i>	Issue	Oper	Comp Result
LD	F6	34+	R2	1	2	3 4
LD	F2	45+	R3	5	6	7 8
MULTD	F0	F2	F4	6	9	19 20
SUBD	F8	F6	F2	7	9	11 12
DIVD	F10	F0	F6	8		
ADDD	F6	F8	F2	13	14	16

Functional unit status:

l unit status:

Time	Name	Busy	Op	dest Fi	S1 Fj	S2 Fk	FU Qj	FU Qk	Fj? Rj	Fk? Rk
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	Yes	Add	F6	F8	F2			No	No
	Divide	Yes	Div	F10	F0	F6			Yes	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
20	<i>FU</i>	Add				Divide				

Scoreboard Example: Cycle 21

Instruction status:

				Read	Exec	Write
Instruction	<i>j</i>	<i>k</i>	Issue	Oper	Comp	Result
LD	F6	34+ R2	1	2	3	4
LD	F2	45+ R3	5	6	7	8
MULTD	F0	F2 F4	6	9	19	20
SUBD	F8	F6 F2	7	9	11	12
DIVD	F10	F0 F6	8	21		
ADDD	F6	F8 F2	13	14	16	

Functional unit status:

l unit status:

Time	Name	Busy	Op	dest Fi	S1 Fj	S2 Fk	FU Qj	FU Qk	Fj? Rj	Fk? Rk
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	Yes	Add	F6	F8	F2			No	No
	Divide	Yes	Div	F10	F0	F6			Yes	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
21	FU				Add		Divide			

Scoreboard Example: Cycle 21

Instruction status:

Instruction		<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Read Oper</i>	<i>Exec Comp</i>	<i>Write Result</i>
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9	19	20
SUBD	F8	F6	F2	7	9	11	12
DIVD	F10	F0	F6	8	21		
ADDD	F6	F8	F2	13	14	16	

- WAR Hazard is now gone...

Functional unit status:

<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>dest Fi</i>	<i>S1 Fj</i>	<i>S2 Fk</i>	<i>FU Qj</i>	<i>FU Qk</i>	<i>Fj? Rj</i>	<i>Fk? Rk</i>
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	Yes	Add	F6	F8	F2			No	No
	Divide	Yes	Div	F10	F0	F6			Yes	Yes

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
21	FU				Add		Divide			

Instruction status:

Instruction status:

Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Read Oper</i>	<i>Exec Comp</i>	<i>Write Result</i>	
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9	19	20
SUBD	F8	F6	F2	7	9	11	12
DIVD	F10	F0	F6	8	21		
ADDD	F6	F8	F2	13	14	16	22

Functional unit status:

l unit status:

Time	Name	Busy	Op	dest Fi	S1 Fj	S2 Fk	FU Qj	FU Qk	Fj? Rj	Fk? Rk
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	No								
39	Divide	Yes	Div	F10	F0	F6			No	No

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
22	FU	Divide								

Scoreboard Example: Cycle 23-60

**Faster than light computation
(skip a couple of cycles)**

Instruction status:

<i>Instruction status:</i>				<i>Read</i>	<i>Exec</i>	<i>Write</i>	
Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Oper</i>	<i>Comp</i>	<i>Result</i>	
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9	19	20
SUBD	F8	F6	F2	7	9	11	12
DIVD	F10	F0	F6	8	21	61	
ADDD	F6	F8	F2	13	14	16	22

Functional unit status:

l unit status:

Time	Name	Busy	Op	dest Fi	S1 Fj	S2 Fk	FU Qj	FU Qk	Fj? Rj	Fk? Rk
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	No								
0	Divide	Yes	Div	F10	F0	F6			No	No

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
61	FU	Divide								

Scoreboard Example: Cycle 62

Instruction status:

<i>Instruction status:</i>				<i>Read</i>	<i>Exec</i>	<i>Write</i>	
Instruction	<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Oper</i>	<i>Comp</i>	<i>Result</i>	
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9	19	20
SUBD	F8	F6	F2	7	9	11	12
DIVD	F10	F0	F6	8	21	61	62
ADDD	F6	F8	F2	13	14	16	22

Functional unit status:

<i>l unit status:</i>		<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>		
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
62	FU									

Instruction status:

<i>Instruction status:</i>					<i>Read</i>	<i>Exec</i>	<i>Write</i>
Instruction		<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Oper</i>	<i>Comp</i>	<i>Result</i>
LD	F6	34+	R2	1	2	3	4
LD	F2	45+	R3	5	6	7	8
MULTD	F0	F2	F4	6	9	19	20
SUBD	F8	F6	F2	7	9	11	12
DIVD	F10	F0	F6	8	21	61	62
ADDD	F6	F8	F2	13	14	16	22

Functional unit status:

<i>l unit status:</i>				<i>dest</i>	<i>S1</i>	<i>S2</i>	<i>FU</i>	<i>FU</i>	<i>Fj?</i>	<i>Fk?</i>
<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Fi</i>	<i>Fj</i>	<i>Fk</i>	<i>Qj</i>	<i>Qk</i>	<i>Rj</i>	<i>Rk</i>
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

Register result status:

Clock		$F0$	$F2$	$F4$	$F6$	$F8$	$F10$	$F12$	...	$F30$
62	FU									

Performance Improvement and Cost

CDC6600

Performance Improvement	Language
1.6X	Fortran
2.5X	Hand-Coded Assembly

Condition:

- No software pipeline scheduling
- No memory
- No caches

The scoreboard on the CDC 6600 had about as much logic as one of the functional units, which is surprisingly low.

A Dynamic Algorithm: Tomasulo's Algorithm

For IBM 360/91 (before caches!)

Goal: High Performance without special compilers

Small number of floating point registers (4 in 360) prevented interesting compiler scheduling of operations

This led Tomasulo to try to figure out how to get more effective registers —
renaming in hardware!

Why Study 1966 Computer?

The descendants of this have flourished!

Alpha 21264, HP 8000, MIPS 10000, Pentium III, PowerPC 604, ...

A Dynamic Algorithm: Tomasulo's Algorithm

Control & buffers **distributed** with Function Units (FU)

- FU buffers called “**reservation stations**”; have pending operands

Registers in instructions replaced by values or pointers to reservation stations(RS); called **register renaming**;

- avoids WAR, WAW hazards
- More reservation stations than registers, so can do optimizations compilers can't

Results to FU from RS, **not through registers**, over **Common Data Bus** that broadcasts results to all FUs

Load and Stores treated as FUs with RSs as well

Integer instructions can go past branches, allowing FP ops beyond basic block in FP queue

Some History - IBM 360/91

Fast 360 for scientific code

Completed in 1967

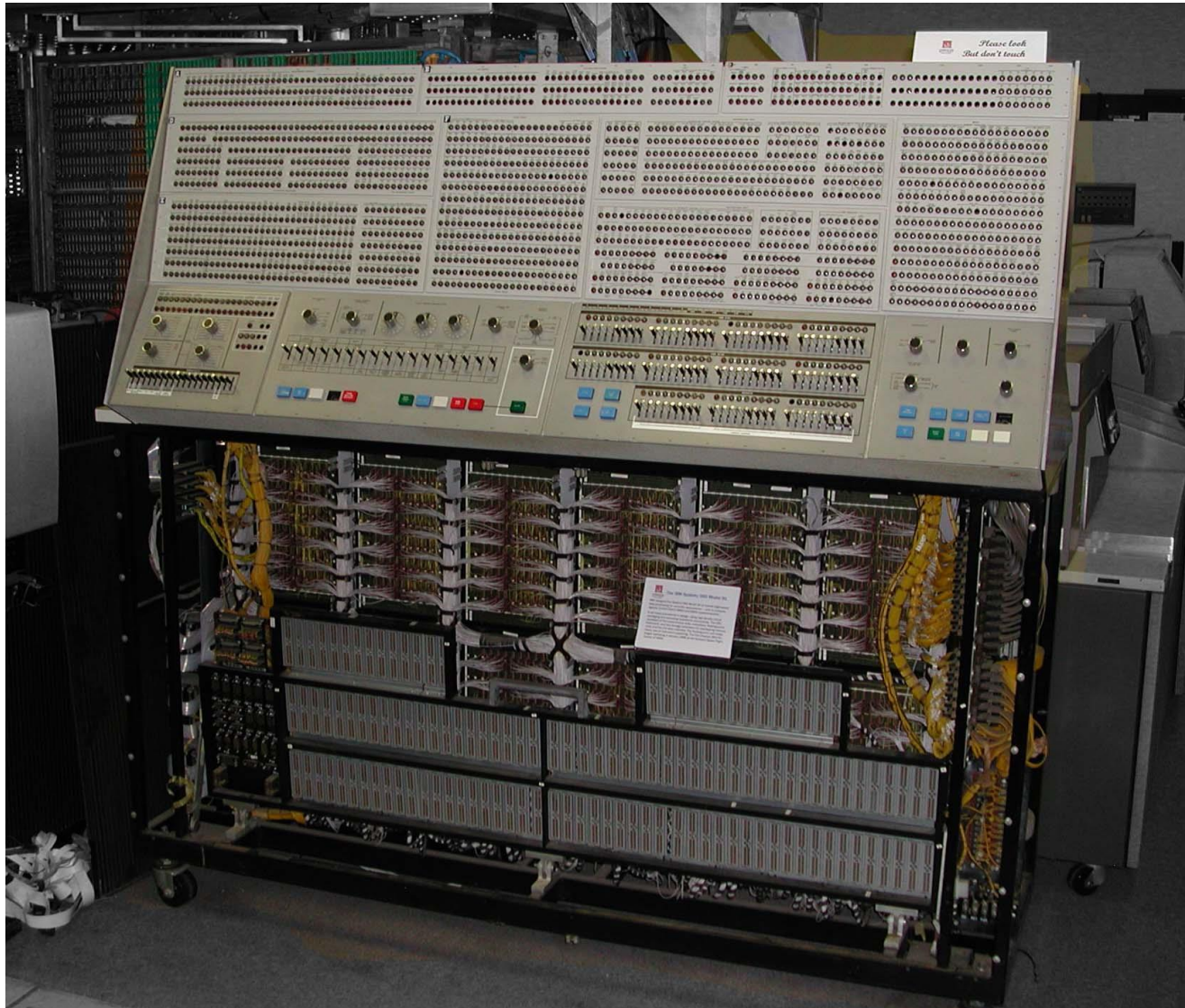
Predates cache memories

Pipelined, rather than multiple, functional units (FU)

We will assume multiple functional units

360 had register memory instructions, we don't

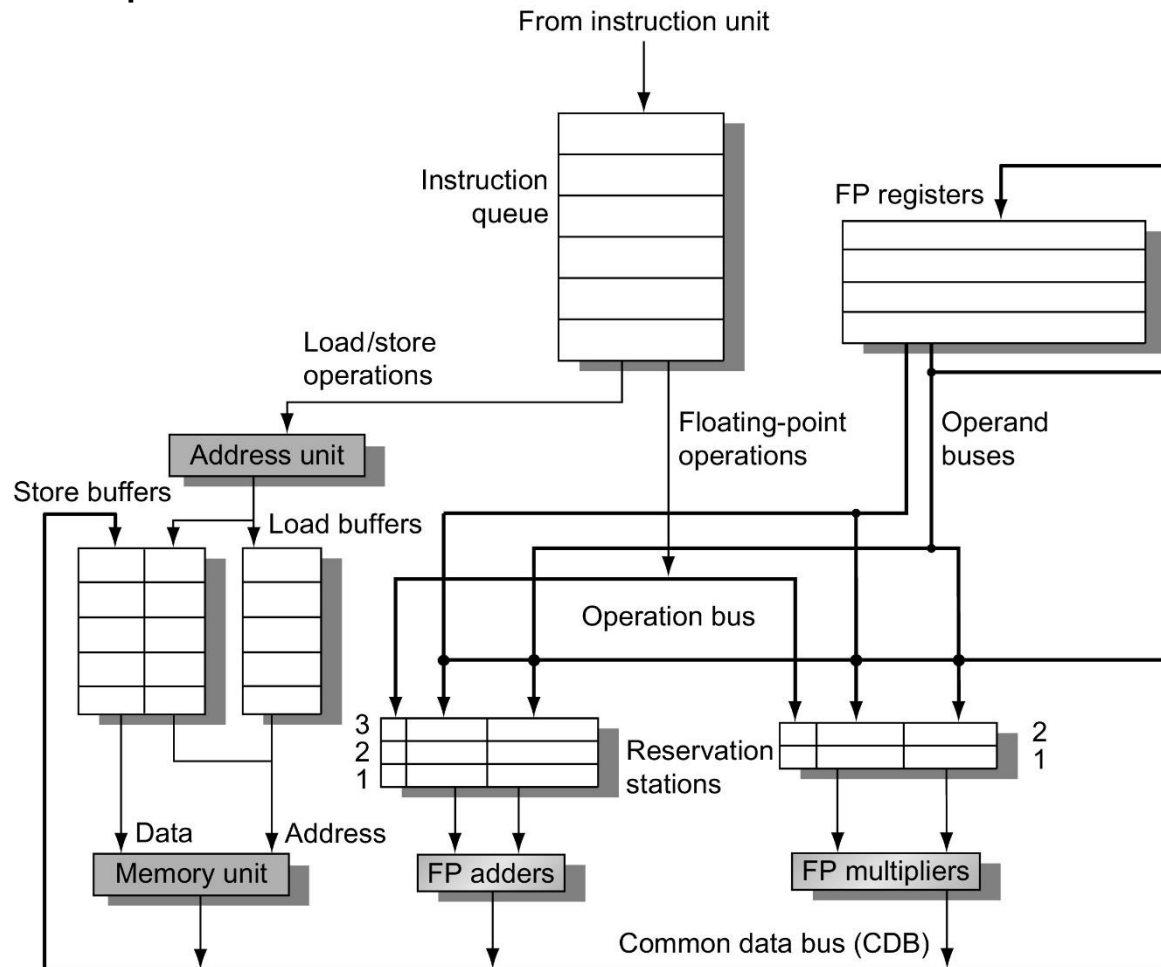
IBM 360/91



Tomasulo Organization

Extend simple pipeline as example for Tomasulo's algorithm

Assume multiple FUs



Register Renaming - Tomasulo's Algorithm

Tomasulo's algm uses *reservation stations* for register renaming

Instruction is “issued” to a reservation station

A pending operand is designated via a tag

Tag = reservation station that will provide the operand

Reservation station with pending instruction fetches and buffers the operand when it becomes available

All FUs place output on the *common data bus* (CDB) with tag

Waiting reservation station gets the data from the CDB (register bypass)

Three Stages of Tomasulo Pipeline

3-stage Execution (ignore IF and MEM)

1. Issue -- Get instruction from FP Op Queue

ALU Op: Check for available reservation station

Load/Store: Check for available load/store buffer

If not, stall due to structural hazard

If reservation station free (no structural hazard), control issues instr & sends operands (renames registers).

2. Execute -- operate on operands (EX)

If operands available, execute operation

If not, monitor CDB for operand

3. Write -- finish execution (WB)

If CDB available, write it on CDB If not, stall

Write on Common Data Bus to all awaiting units;

mark reservation station available;

Normal data bus: data + destination (“go to” bus)

Common data bus: data + **source** (“**come from**” bus)

- 64 bits of data + 4 bits of Functional Unit source address
- Write if matches expected Functional Unit (produces result)
- Does the broadcast

Tomasulo Pipeline, cont

Reservation Stations

Handle distributed hazard detection and instruction control

Everything, except store buffers, has a *tag*

4-bit tag specifies reservation station or load buffer

Specifies which FU will produce result

Register specifier is used to assign tags

THEN IT'S DISCARDED!

Register specifiers are ONLY used in ISSUE

Tomasulo Pipeline, cont

Reservation Stations

Op	Opcode to perform in the unit (e.g., + or −)
Q_j, Q_k	Reservation stations producing source registers (value to be written)
V_j, V_k	Value of Source operands • Note: $Q_j, Q_k = 0 \Rightarrow$ ready • Store buffers only have Q_i for RS producing result
Busy	Indicates reservation station or FU is busy

Register File

Q_i	Tag Field
Busy	Currently in use

Load and Store Buffers

Busy	Currently in use
A	Memory Address

Register result status: Indicates which functional unit will write each register, if one exists. Blank when no pending instructions that will write that register.

Tomasulo Example

Example code

L.D F6, 34 (R2)

L.D F2, 45 (R3)

MULT.D F0, F2, F4

SUB.D F8, F6, F2

DIV.D F10, F0, F6

ADD.D F6, F8, F2

Example speed:

2 clocks for Fl .pt. +,-; 10 for * ; 40 clks for /; 2 for ld, sd;

Tomasulo Example

Instruction stream

Instruction status:

Instruction		<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result
LD	F6	34+	R2			
LD	F2	45+	R3			
MULTD	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

	Busy	Address
Load1	No	
Load2	No	
Load3	No	

3 Load/Buffers

Reservation Stations:

Time	Name	Busy	Op	S1 Vj	S2 Vk	RS Qj	RS Qk
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
	Mult2	No					

FU count
down

3 FP Adder R.S.
2 FP Mult R.S.

Register result status:

Clock

0

	F0	F2	F4	F6	F8	F10	F12	...	F30
FU									

Clock cycle
counter

Tomasulo Example: Cycle 1

Instruction status:

Instruction		<i>j</i>	<i>k</i>	Issue	Exec	Write
LD	F6	34+	R2	1		
LD	F2	45+	R3			
MULTD	F0	F2	F4			
SUBD	F8	F6	F2			
DIVD	F10	F0	F6			
ADDD	F6	F8	F2			

	Busy	Address
Load1	Yes	34+R2
Load2	No	
Load3	No	

Reservation Stations:

			<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i> <i>Qk</i>
	Add1	No				
	Add2	No				
	Add3	No				
	Mult1	No				
	Mult2	No				

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
1				Load1					

Tomasulo Example: Cycle 2

Instruction status:

				Exec	Write		
Instruction		<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy Address
LD	F6	34+	R2	1			Load1 Yes 34+R2
LD	F2	45+	R3	2			Load2 Yes 45+R3
MULTD	F0	F2	F4				Load3 No
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
Add1		No					
Add2		No					
Add3		No					
Mult1		No					
Mult2		No					

Register result status:

		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
Clock	2	FU	Load2		Load1					

Note: Can have multiple loads outstanding

Tomasulo Example: Cycle 3

Instruction status:

				Exec	Write		
Instruction	<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	Load1	Yes 34+R2
LD	F2	45+	R3	2		Load2	Yes 45+R3
MULTD	F0	F2	F4	3		Load3	No
SUBD	F8	F6	F2				
DIVD	F10	F0	F6				
ADDD	F6	F8	F2				

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	Yes	MULTD		R(F4)	Load2	
	Mult2	No					

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
3	FU	Mult1	Load2		Load1				

- Note: registers names are removed ("renamed") in Reservation Stations; MULT issued
- Load1 completing; what is waiting for Load1?

Tomasulo Example: Cycle 4

Instruction status:

				Issue	Exec Comp	Write Result		
Instruction	<i>j</i>	<i>k</i>					Busy	Address
LD	F6	34+	R2	1	3	4	Load1	No
LD	F2	45+	R3	2	4		Load2	Yes 45+R3
MULTD	F0	F2	F4	3			Load3	No
SUBD	F8	F6	F2	4				
DIVD	F10	F0	F6					
ADDD	F6	F8	F2					

Reservation Stations:

Time	Name	Busy	Op	S1 Vi	S2 Vk	RS Qi	RS Ok
Add1		Yes	SUBD	M(A1)			Load2
Add2		No					
Add3		No					
Mult1		Yes	MULTD		R(F4)		Load2
Mult2		No					

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
4	FU								
	Mult1	Load2		M(A1)	Add1				

- Load2 completing; what is waiting for Load2?

Tomasulo Example: Cycle 5

Instruction status:

				Exec	Write		
Instruction	<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1
LD	F2	45+	R3	2	4	5	Load2
MULTD	F0	F2	F4	3			Load3
SUBD	F8	F6	F2	4			
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2				

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
2	Add1	Yes	SUBD	M(A1)	M(A2)		
	Add2	No					
	Add3	No					
10	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
5	FU	Mult1	M(A2)		M(A1)	Add1	Mult2		

- Timer starts down for Add1, Mult1

Tomasulo Example: Cycle 6

Instruction status:

				Issue	Exec Comp	Write Result		
Instruction		j	k				Busy	Address
LD	F6	34+	R2	1	3	4	Load1	No
LD	F2	45+	R3	2	4	5	Load2	No
MULTD	F0	F2	F4	3			Load3	No
SUBD	F8	F6	F2	4				
DIVD	F10	F0	F6	5				
ADDD	F6	F8	F2	6				

Reservation Stations:

				S1	S2	RS	RS
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
1	Add1	Yes	SUBD	M(A1)	M(A2)		
	Add2	Yes	ADDD		M(A2)	Add1	
	Add3	No					
9	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
6									
FU	Mult1	M(A2)		Add2	Add1	Mult2			

- Issue ADDD here despite name dependency on F6?

Tomasulo Example: Cycle 7

Instruction status:

				Exec	Write		
Instruction	<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1
LD	F2	45+	R3	2	4	5	Load2
MULTD	F0	F2	F4	3			Load3
SUBD	F8	F6	F2	4	7		
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2	6			

Reservation Stations:

			<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	V _j	V _k	Q _j Q _k
0	Add1	Yes	SUBD	M(A1)	M(A2)	
	Add2	Yes	ADDD		M(A2)	Add1
	Add3	No				
8	Mult1	Yes	MULTD	M(A2)	R(F4)	
	Mult2	Yes	DIVD		M(A1)	Mult1

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
7	FU								
	Mult1	M(A2)		Add2	Add1	Mult2			

- Add1 (SUBD) completing; what is waiting for it?

Tomasulo Example: Cycle 8

Instruction status:

				Exec		Write		
Instruction		<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1	No
LD	F2	45+	R3	2	4	5	Load2	No
MULTD	F0	F2	F4	3			Load3	No
SUBD	F8	F6	F2	4	7	8		
DIVD	F10	F0	F6	5				
ADDD	F6	F8	F2	6				

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
2	Add2	Yes	ADDD	(M-M)	M(A2)		
	Add3	No					
7	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
8	FU	Mult1	M(A2)		Add2	(M-M)	Mult2			

Tomasulo Example: Cycle 9

Instruction status:

				Exec	Write		
Instruction	<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1
LD	F2	45+	R3	2	4	5	Load2
MULTD	F0	F2	F4	3			Load3
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2	6			

Reservation Stations:

				S1	S2	RS	RS
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
	Add1	No					
1	Add2	Yes	ADDD	(M-M)	M(A2)		
	Add3	No					
6	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
9	FU	Mult1	M(A2)		Add2	(M-M)	Mult2		

Tomasulo Example: Cycle 10

Instruction status:

				Exec	Write		
Instruction	<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1
LD	F2	45+	R3	2	4	5	Load2
MULTD	F0	F2	F4	3			Load3
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2	6	10		

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
	Add1	No					
0	Add2	Yes	ADDD	(M-M)	M(A2)		
	Add3	No					
5	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
10	FU	Mult1	M(A2)		Add2	(M-M)	Mult2		

- Add2 (ADDD) completing; what is waiting for it?

Tomasulo Example: Cycle 11

Instruction status:

				Exec	Write		
Instruction		j	k	Issue	Comp	Result	Busy Address
LD	F6	34+	R2	1	3	4	Load1 No
LD	F2	45+	R3	2	4	5	Load2 No
MULTD	F0	F2	F4	3			Load3 No
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2	6	10	11	

Reservation Stations:

on Stations:

				S1	S2	RS	RS
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
	Add1	No					
	Add2	No					
	Add3	No					
4	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
11	FU	Mult1	M(A2)	(M-M+M)	(M-M)	Mult2			

- Write result of ADDD here?
- All quick instructions complete in this cycle!

Tomasulo Example: Cycle 12

Instruction status:

				Exec	Write		
Instruction		j	k	Issue	Comp	Result	Busy Address
LD	F6	34+	R2	1	3	4	Load1 No
LD	F2	45+	R3	2	4	5	Load2 No
MULTD	F0	F2	F4	3			Load3 No
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2	6	10	11	

Reservation Stations:

on Stations:

				S1	S2	RS	RS
Time	Name	Busy	Op	Vj	Vk	Qj	Qk
	Add1	No					
	Add2	No					
	Add3	No					
3	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
12	FU								
	Mult1	M(A2)		(M-M+M	(M-M)	Mult2			

Tomasulo Example: Cycle 13

Instruction status:

Instruction		<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Exec Comp</i>	<i>Write Result</i>		<i>Busy</i>	<i>Address</i>
LD	F6	34+	R2	1	3	4	Load1	No	
LD	F2	45+	R3	2	4	5	Load2	No	
MULTD	F0	F2	F4	3			Load3	No	
SUBD	F8	F6	F2	4	7	8			
DIVD	F10	F0	F6	5					
ADDD	F6	F8	F2	6	10	11			

Reservation Stations:

<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>S1 Vj</i>	<i>S2 Vk</i>	<i>RS Qj</i>	<i>RS Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
2	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
13	FU	Mult1	M(A2)		(M-M+M	(M-M)	Mult2			

Tomasulo Example: Cycle 14

Instruction status:

				Exec	Write		
Instruction	j	k	Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1
LD	F2	45+	R3	2	4	5	Load2
MULTD	F0	F2	F4	3			Load3
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2	6	10	11	

Reservation Stations:

		S1	S2	RS	RS
Time	Name	Busy	Op	Vj	Vk
	Add1	No			
	Add2	No			
	Add3	No			
1	Mult1	Yes	MULTD	M(A2)	R(F4)
	Mult2	Yes	DIVD		M(A1)

Register result status:

Clock	F0	F2	F4	F6	F8	F10	F12	...	F30
14	FU	Mult1	M(A2)		(M-M+M	(M-M)	Mult2		

Tomasulo Example: Cycle 15

Instruction status:

				Exec	Write		
Instruction		<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy Address
LD	F6	34+	R2	1	3	4	Load1 No
LD	F2	45+	R3	2	4	5	Load2 No
MULTD	F0	F2	F4	3	15		Load3 No
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5			
ADDD	F6	F8	F2	6	10	11	

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
0	Mult1	Yes	MULTD	M(A2)	R(F4)		
	Mult2	Yes	DIVD		M(A1)	Mult1	

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
15	FU	Mult1	M(A2)		(M-M+M	(M-M)	Mult2			

- Mult1 (MULTD) completing; what is waiting for it?

Tomasulo Example: Cycle 16

Instruction status:

Instruction		<i>j</i>	<i>k</i>	Issue	Exec Comp	Write Result		Busy	Address
LD	F6	34+	R2	1	3	4	Load1	No	
LD	F2	45+	R3	2	4	5	Load2	No	
MULTD	F0	F2	F4	3	15	16	Load3	No	
SUBD	F8	F6	F2	4	7	8			
DIVD	F10	F0	F6	5					
ADDD	F6	F8	F2	6	10	11			

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> V _j	<i>S2</i> V _k	<i>RS</i> Q _j	<i>RS</i> Q _k
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
40	Mult2	Yes	DIVD	M*F4	M(A1)		

Register result status:

Clock		F0	F2	F4	F6	F8	F10	F12	...	F30
16	FU	M*F4	M(A2)		(M-M+M	(M-M)	Mult2			

- Just waiting for Mult2 (DIVD) to complete

Tomasulo Example: Cycle 17-54

***Faster than light computation
(skip a couple of cycles)***

Tomasulo Example: Cycle 55

Instruction status:

Instruction		<i>j</i>	<i>k</i>	<i>Issue</i>	<i>Exec Comp</i>	<i>Write Result</i>		<i>Busy</i>	<i>Address</i>
LD	F6	34+	R2	1	3	4	Load1	No	
LD	F2	45+	R3	2	4	5	Load2	No	
MULTD	F0	F2	F4	3	15	16	Load3	No	
SUBD	F8	F6	F2	4	7	8			
DIVD	F10	F0	F6	5					
ADDD	F6	F8	F2	6	10	11			

Reservation Stations:

<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>S1 Vj</i>	<i>S2 Vk</i>	<i>RS Qj</i>	<i>RS Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
1	Mult2	Yes	DIVD	M*F4	M(A1)		

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
55	FU	M*F4	M(A2)		(M-M+M	(M-M)	Mult2			

Tomasulo Example: Cycle 56

Instruction status:

				Exec	Write		
Instruction	<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy	Address
LD	F6	34+	R2	1	3	4	Load1
LD	F2	45+	R3	2	4	5	Load2
MULTD	F0	F2	F4	3	15	16	Load3
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5	56		
ADDD	F6	F8	F2	6	10	11	

Reservation Stations:

Time	Name	Busy	Op	<i>S1</i> <i>Vj</i>	<i>S2</i> <i>Vk</i>	<i>RS</i> <i>Qj</i>	<i>RS</i> <i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
0	Mult2	Yes	DIVD	M*F4	M(A1)		

Register result status:

Clock	<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
56	<i>FU</i>	M*F4	M(A2)		(M-M+N	(M-M)	Mult2		

- Mult2 (DIVD) is completing; what is waiting for it?

Tomasulo Example: Cycle 57

Instruction status:

				Exec	Write		
Instruction		<i>j</i>	<i>k</i>	Issue	Comp	Result	Busy Address
LD	F6	34+	R2	1	3	4	Load1 No
LD	F2	45+	R3	2	4	5	Load2 No
MULTD	F0	F2	F4	3	15	16	Load3 No
SUBD	F8	F6	F2	4	7	8	
DIVD	F10	F0	F6	5	56	57	
ADDD	F6	F8	F2	6	10	11	

Reservation Stations:

				<i>S1</i>	<i>S2</i>	<i>RS</i>	<i>RS</i>
Time	Name	Busy	Op	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
	Mult2	Yes	DIVD	M*F4	M(A1)		

Register result status:

Clock		<i>F0</i>	<i>F2</i>	<i>F4</i>	<i>F6</i>	<i>F8</i>	<i>F10</i>	<i>F12</i>	...	<i>F30</i>
56	FU	M*F4	M(A2)			(M-M+M	(M-M)	Result		

- Once again: In-order issue, out-of-order execution and out-of-order completion.

Tomasulo, cont.

Advantages

- Distribution of hazard detection
- Elimination of WAR and WAW stalls

Common Data Bus

- + Broadcasts results to multiple instructions, bypasses registers
- Central bottleneck
 - Could duplicate (increases required hardware)

Register Renaming

- + Eliminates WAR and WAW Hazards
- + Allows dynamic loop unrolling
 - Especially important with only 4 registers
- Requires many associative lookups

Tomasulo, cont**

Out-of-order loads and stores?

What about WAW, RAW, and WAR hazards?

Compare all load addresses w/ address in store buffers

Compare all store addresses w/ address in load/store buffers

Stall if they match

CDB is a bottleneck

Could duplicate

Increases the required hardware

Complex implementation

Tomasulo Drawbacks

Complexity

- delays of 360/91, MIPS 10000, Alpha 21264, IBM PPC 620 in CA:AQA 2/e, but not in silicon!

Many associative stores (CDB) at high speed

Performance limited by Common Data Bus

- Each CDB must go to multiple functional units
⇒ high capacitance, high wiring density
- Number of functional units that can complete per cycle limited to one!
Multiple CDBs ⇒ more FU logic for parallel assoc stores

Non-precise interrupts!

We will address this later

Loops with Tomasulo's Algorithm

Example

Loop: LD	F0	0	R1
MULTD	F4	F0	F2
SD	F4	0	R1
SUBI	R1	R1	#8
BNEZ	R1	Loop	

- This time assume Multiply takes 4 clocks
- Assume 1st load takes 8 clocks
(L1 cache miss), 2nd load takes 1 clock (hit)
- To be clear, will show clocks for SUBI, BNEZ
- Reality: integer instructions ahead of Fl. Pt. Instructions
- Show 2 iterations

Loop Example

Instruction status:

Instruction status:						Exec Write				
ITER	Instruction	j	k	Issue	CompResult		Busy	Addr	Fu	
Iteration Count	1	LD	F0	0	R1		Load1	No		
	1	MULTD	F4	F0	F2		Load2	No		
	1	SD	F4	0	R1		Load3	No		
	2	LD	F0	0	R1		Store1	No		
	2	MULTD	F4	F0	F2		Store2	No		
	2	SD	F4	0	R1		Store3	No		

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:
	Add1	No						LD
	Add2	No						MULTD
	Add3	No						SD
	Mult1	No						SUBI
	Mult2	No						BNEZ

Added Store Buffers

Code:

LD	F0	0	R1
MULTD	F4	F0	F2
SD	F4	0	R1
SUBI	R1	R1	#8
BNEZ	R1	Loop	

Instruction Loop

Register result status

Clock	R1	Fu	F0	F2	F4	F6	F8	F10	F12	...	F30
0	80										

Value of Register used for address, iteration control

Loop Example: Cycle 1

Instruction status:

						Exec Write			
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1		Load1	Yes	80
							Load2	No	
							Load3	No	
							Store1	No	
							Store2	No	
							Store3	No	

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:	
	Add1	No						LD	F0 0 R1
	Add2	No						MULTD	F4 F0 F2
	Add3	No						SD	F4 0 R1
	Mult1	No						SUBI	R1 R1 #8
	Mult2	No						BNEZ	R1 Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
1	80	Load1								

Loop Example: Cycle 2

Instruction status:

					Exec Write				
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1		Load1	Yes	80
1	MULTD	F4	F0	F2	2		Load2	No	
							Load3	No	
							Store1	No	
							Store2	No	
							Store3	No	

Reservation Stations:

					S1		S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk		Code:			
	Add1	No							LD	F0	0	R1
	Add2	No							MULTD	F4	F0	F2
	Add3	No							SD	F4	0	R1
	Mult1	Yes	Multd			R(F2)	Load1		SUBI	R1	R1	#8
	Mult2	No							BNEZ	R1	Loop	

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
2	80	Fu	Load1	Mult1						

Loop Example: Cycle 4

Instruction status:

						Exec Write			
ITER	Instruction	j	k	Issue	CompResult	Busy	Addr	Fu	
1	LD	F0	0	R1	1	Load1	Yes	80	
1	MULTD	F4	F0	F2	2	Load2	No		
1	SD	F4	0	R1	3	Load3	No		
						Store1	Yes	80	Mult1
						Store2	No		
						Store3	No		

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:	
	Add1	No						LD	F0 0 R1
	Add2	No						MULTD	F4 F0 F2
	Add3	No						SD	F4 0 R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI	R1 R1 #8
	Mult2	No						BNEZ	R1 Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
4	80	Fu	Load1	Mult1						

Dispatching SUBI Instruction (not in FP queue)

Loop Example: Cycle 5

Instruction status:

Instruction status:

			Exec Write						
ITER	Instruction		j	k	Issue	CompResult	Busy	Addr	Fu
1	LD	F0	0	R1	1		Load1	Yes	80
1	MULTD	F4	F0	F2	2		Load2	No	
1	SD	F4	0	R1	3		Load3	No	
							Store1	Yes	80
							Store2	No	
							Store3	No	
									Mult1

Reservation Stations:

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:				
	Add1	No						LD	F0	0		R1
	Add2	No						MULTD	F4	F0		F2
	Add3	No						SD	F4	0		R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI	R1	R1		#8
	Mult2	No						BNEZ	R1	Loop		

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
5	72	Load1		Mult1						

And, BNEZ instruction (not in FP queue)

Loop Example: Cycle 6

Instruction status:

						Exec Write					
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu		
1	LD	F0	0	R1	1		Load1	Yes	80		
1	MULTD	F4	F0	F2	2		Load2	Yes	72		
1	SD	F4	0	R1	3		Load3	No			
2	LD	F0	0	R1	6		Store1	Yes	80	Mult1	
							Store2	No			
							Store3	No			

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI	R1	R1	#8
	Mult2	No						BNEZ	R1	Loop	

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
6	72	Fu	Load2							
				Mult1						

Notice that F0 never sees Load from location 80

Loop Example: Cycle 7

Instruction status:

instruction status:

		Exec Write								
ITER	Instruction	j	k	Issue	CompResult		Busy	Addr	Fu	
1	LD	F0	0	R1	1	Load1	Yes	80		
1	MULTD	F4	F0	F2	2		Load2	Yes		72
1	SD	F4	0	R1	3		Load3	No		
2	LD	F0	0	R1	6	Store1	Yes	80	Mult1	
2	MULTD	F4	F0	F2	7	Store2	No			
						Store3	No			

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:
	Add1	No						LD F0 0 R1
	Add2	No						MULTD F4 F0 F2 ←
	Add3	No						SD F4 0 R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI R1 R1 #8
	Mult2	Yes	Multd		R(F2)	Load2		BNEZ R1 Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
7	72	Fu	Load2	Mult2						

Register file completely detached from computation
First and Second iteration completely overlapped

Loop Example: Cycle 8

Instruction status:

					Exec Write				
ITER	Instruction	j	k	Issue	CompResult	Busy	Addr	Fu	
1	LD	F0	0	R1	1	Load1	Yes	80	
1	MULTD	F4	F0	F2	2	Load2	Yes	72	
1	SD	F4	0	R1	3	Load3	No		
2	LD	F0	0	R1	6	Store1	Yes	80	Mult1
2	MULTD	F4	F0	F2	7	Store2	Yes	72	Mult2
2	SD	F4	0	R1	8	Store3	No		

Reservation Stations:

				S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0 R1
	Add2	No						MULTD	F4	F0 F2
	Add3	No						SD	F4	0 R1
	Mult1	Yes	Multd		R(F2)	Load1		SUBI	R1	R1 #8
	Mult2	Yes	Multd		R(F2)	Load2		BNEZ	R1	Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
8	72	Fu	Load2	Mult2						

Loop Example: Cycle 10

Instruction status:

ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	Load1	No	
1	MULTD	F4	F0	F2	2		Load2	Yes	72
1	SD	F4	0	R1	3		Load3	No	
2	LD	F0	0	R1	6	10	Store1	Yes	80
2	MULTD	F4	F0	F2	7		Store2	Yes	72
2	SD	F4	0	R1	8		Store3	No	

Reservation Stations:

Reservation Stations:					S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
4	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1	#8
	Mult2	Yes	Multd		R(F2)	Load2		BNEZ	R1	Loop	

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
10	64	Load2		Mult2						

Load2 completing: who is waiting?

Note: Dispatching BNEZ

Loop Example: Cycle 11

Instruction status:

					Exec Write					
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu	
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2			Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80
2	MULTD	F4	F0	F2	7			Store2	Yes	72
2	SD	F4	0	R1	8			Store3	No	
									Mult1	
									Mult2	

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0
	Add2	No						MULTD	F4	F0
	Add3	No						SD	F4	0
3	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1
4	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
11	64	Fu	Load3			Mult2				

Next load in sequence

Loop Example: Cycle 12

Instruction status:

					Exec Write					
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu	
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2			Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80
2	MULTD	F4	F0	F2	7			Store2	Yes	72
2	SD	F4	0	R1	8			Store3	No	
										Mult1
										Mult2

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0
	Add2	No						MULTD	F4	F0
	Add3	No						SD	F4	0
2	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1
3	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
12	64	Fu	Load3	Mult2						

Why not issue third multiply?

Loop Example: Cycle 13

Instruction status:

					Exec Write					
ITER	Instruction		j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2			Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80
2	MULTD	F4	F0	F2	7			Store2	Yes	72
2	SD	F4	0	R1	8			Store3	No	

Reservation Stations:

				S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0 R1
	Add2	No						MULTD	F4	F0 F2
	Add3	No						SD	F4	0 R1
1	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1 #8
2	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop

Register result status

Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
13	64	Fu	Load3	Mult2							

Why not issue third store?

Loop Example: Cycle 14

Instruction status:

					Exec Write				
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	Load1	No	
1	MULTD	F4	F0	F2	2	14	Load2	No	
1	SD	F4	0	R1	3		Load3	Yes	64
2	LD	F0	0	R1	6	10	Store1	Yes	80
2	MULTD	F4	F0	F2	7		Store2	Yes	72
2	SD	F4	0	R1	8		Store3	No	
									Mult1
									Mult2

Reservation Stations:

				S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0
	Add2	No						MULTD	F4	F0
	Add3	No						SD	F4	0
0	Mult1	Yes	Multd	M[80]	R(F2)			SUBI	R1	R1
1	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
14	64	Fu	Load3	Mult2						

Mult1 completing. Who is waiting?

Loop Example: Cycle 15

Instruction status:

					Exec Write					
ITER	Instruction		j	k	Issue	Comp	Result	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2	14	15	Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80
2	MULTD	F4	F0	F2	7	15		Store2	Yes	72
2	SD	F4	0	R1	8			Store3	No	

Reservation Stations:

				S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:		
	Add1	No						LD	F0	0
	Add2	No						MULTD	F4	F0
	Add3	No						SD	F4	0
	Mult1	No						SUBI	R1	R1
0	Mult2	Yes	Multd	M[72]	R(F2)			BNEZ	R1	Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
15	64	Fu	Load3	Mult2						

Mult2 completing. Who is waiting?

Loop Example: Cycle 16

Instruction status:

ITER	Instruction	j	k	Exec Write			Busy	Addr	Fu
				Issue	Comp	Result			
1	LD	F0	0	R1	1	9	10	Load1	No
1	MULTD	F4	F0	F2	2	14	15	Load2	No
1	SD	F4	0	R1	3			Load3	Yes
2	LD	F0	0	R1	6	10	11	Store1	Yes
2	MULTD	F4	F0	F2	7	15	16	Store2	Yes
2	SD	F4	0	R1	8			Store3	No

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:				
	Add1	No						LD	F0	0	R1	
	Add2	No						MULTD	F4	F0	F2	
	Add3	No						SD	F4	0	R1	
4	Mult1	Yes	Multd		R(F2)	Load3		SUBI	R1	R1	#8	
	Mult2	No						BNEZ	R1	Loop		

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
16	64	Fu	Load3		Mult1					

Loop Example: Cycle 17

Instruction status:

					Exec Write					
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu	
1	LD	F0	0	R1	1	9	10	Load1	No	
1	MULTD	F4	F0	F2	2	14	15	Load2	No	
1	SD	F4	0	R1	3			Load3	Yes	64
2	LD	F0	0	R1	6	10	11	Store1	Yes	80
2	MULTD	F4	F0	F2	7	15	16	Store2	Yes	72
2	SD	F4	0	R1	8			Store3	Yes	64
										[80]*R2
										[72]*R2
										Mult1

Reservation Stations:

					S1	S2	RS				
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	Add1	No						LD	F0	0	R1
	Add2	No						MULTD	F4	F0	F2
	Add3	No						SD	F4	0	R1
	Mult1	Yes	Multd		R(F2)	Load3		SUBI	R1	R1	#8
	Mult2	No						BNEZ	R1	Loop	

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
17	64	Fu	Load3	Mult1						

Loop Example: Cycle 19

Instruction status:

					Exec Write					
ITER	Instruction	j	k	Issue	Comp	Result	Busy	Addr	Fu	
1	LD F0	0	R1	1	9	10	Load1	No		
1	MULTD F4	F0	F2	2	14	15	Load2	No		
1	SD F4	0	R1	3	18	19	Load3	Yes	64	
2	LD F0	0	R1	6	10	11	Store1	No		
2	MULTD F4	F0	F2	7	15	16	Store2	Yes	72	[72]*R2
2	SD F4	0	R1	8	19		Store3	Yes	64	Mult1

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:
	Add1	No						LD F0 0 R1
	Add2	No						MULTD F4 F0 F2
	Add3	No						SD F4 0 R1
	Mult1	Yes	Multd		R(F2)	Load3		SUBI R1 R1 #8
	Mult2	No						BNEZ R1 Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
19	56	Fu	Load3	Mult1						

Loop Example: Cycle 20

Instruction status:

ITER	Instruction	j	k	Issue	Comp	Result	Exec	Write	Busy	Addr	Fu
1	LD	F0	0	R1	1	9	10	Load1	Yes	56	
1	MULTD	F4	F0	F2	2	14	15	Load2	No		
1	SD	F4	0	R1	3	18	19	Load3	Yes	64	
2	LD	F0	0	R1	6	10	11	Store1	No		
2	MULTD	F4	F0	F2	7	15	16	Store2	No		
2	SD	F4	0	R1	8	19	20	Store3	Yes	64	Mult1

Reservation Stations:

Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:
	Add1	No						LD F0 0 R1
	Add2	No						MULTD F4 F0 F2
	Add3	No						SD F4 0 R1
	Mult1	Yes	Multd		R(F2)	Load3		SUBI R1 R1 #8
	Mult2	No						BNEZ R1 Loop

Register result status

Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
20	56	Fu	Load1	Mult1						

Once again: In-order issue, out-of-order execution and out-of-order completion.

Why can Tomasulo overlap iterations of loops?

Register renaming

- Multiple iterations use different physical destinations for registers (dynamic loop unrolling).

Reservation stations

- Permit instruction issue to advance past integer control flow operations
- Also buffer old values of registers - totally avoiding the WAR stall that we saw in the scoreboard.

Other perspective: Tomasulo building data flow dependency graph on the fly.

Tomasulo's scheme offers 2 major advantages

(1) the distribution of the hazard detection logic

- distributed reservation stations and the CDB
- If multiple instructions waiting on single result, & each instruction has other operand, then instructions can be released simultaneously by broadcast on CDB
- If a centralized register file were used, the units would have to read their results from the registers when register buses are available.

(2) the elimination of stalls for WAW and WAR hazards

What about Precise Interrupts ?

- **Tomasulo had:**

In-order issue, out-of-order execution, and out-of-order completion

- **Need to “fix” the out-of-order completion aspect so that we can find precise breakpoint in instruction stream.**

Relationship between Precise Interrupts and Speculation:

Speculation is a form of guessing.

Important for branch prediction:

- Need to “take our best shot” at predicting branch direction.

If we speculate and are wrong, need to back up and restart execution to point at which we predicted incorrectly:

- This is exactly same as precise exceptions!

Technique for both precise interrupts/exceptions and speculation:
in-order completion or commit

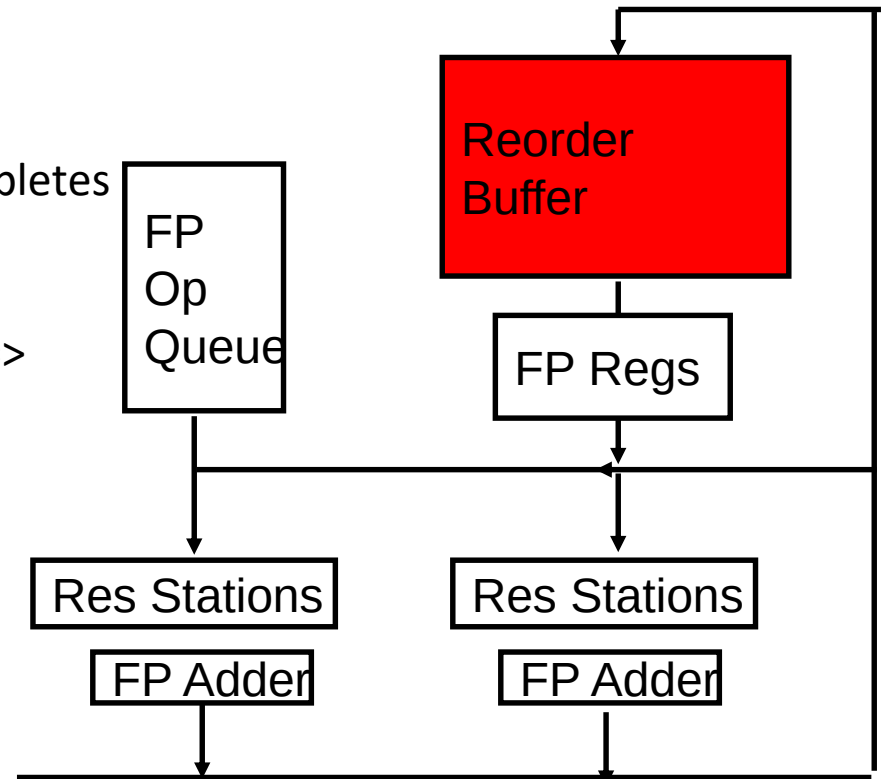
HW Support for Precise Interrupts

Need HW buffer for results of uncommitted instructions:

reorder buffer

- 3 fields: instr, destination, value
- Use reorder buffer number instead of reservation station when execution completes
- Supplies operands between execution complete & commit
- (Reorder buffer can be operand source => more registers like RS)
- Instructions commit
- Once instruction commits, result is put into register
- As a result, easy to undo speculated instructions on mispredicted branches

or exceptions



Four Steps of Speculative Tomasulo Algorithm

1. Issue—get instruction from FP Op Queue

If reservation station **and reorder buffer slot** free, issue instr & send operands **& reorder buffer no. for destination** (this stage sometimes called “dispatch”)

2. Execution—operate on operands (EX)

When both operands ready then execute; if not ready, watch CDB for result; when both in reservation station, execute; checks RAW (sometimes called “issue”)

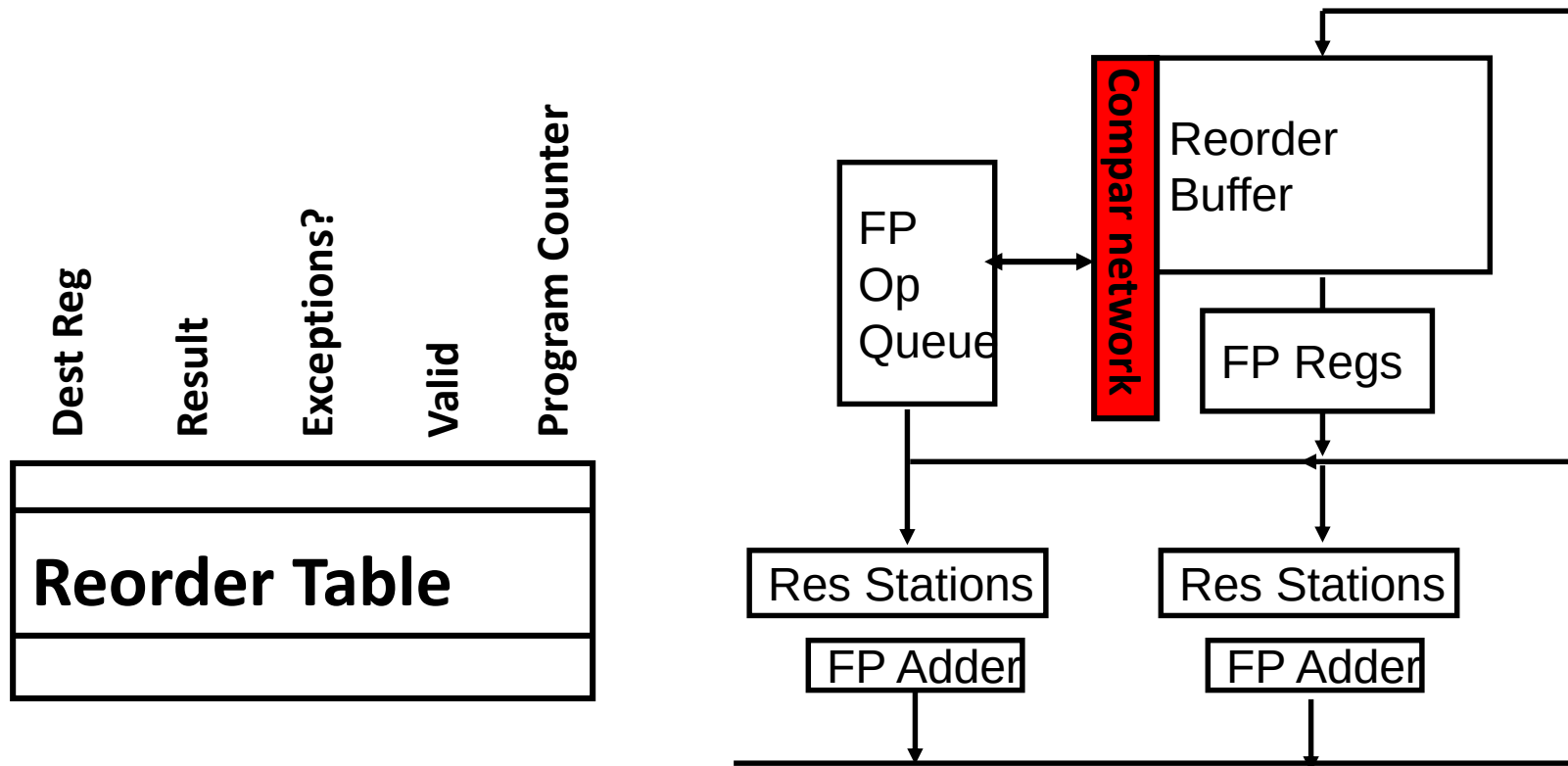
3. Write result—finish execution (WB)

Write on Common Data Bus to all awaiting FUs **& reorder buffer**; mark reservation station available.

4. Commit—update register with reorder result

When instr. at head of reorder buffer & result present, update register with result (or store to memory) and remove instr from reorder buffer. Mispredicted branch flushes reorder buffer (sometimes called “graduation”)

What are the hardware complexities with reorder buffer (ROB)?



How do you find the latest version of a register?

- (As specified by Smith paper) need associative comparison network
- Could use future file or just use the register result status buffer to track which specific reorder buffer has received the value

Need as many ports on ROB as register file

Acknowledgements

These slides contain material developed and copyright by:

- Sarita Adve (UIUC)
- Nathan Beckmann (CMU)
- David Patterson (UC Berkeley)
- Yuanchun Zhang (NUDT)
- Qianni Deng, Yanyan Shen (SJTU)
- Changsheng Xie (HUST)